eman ta zabal zazu
Universidad del País Vasco
Euskal Herriko Unibertsitatea
INFORMATIKA FAKULTATEA
FACULTAD DE INFORMÁTICA

Bachelor Thesis

Bachelor in Computer Science

Software Engineering

A multi-agent system for AI-assisted generation and block-level editing of professional documents

Telmo Goicoechea Jiménez

Advisors
Galdos Otermin, Aritz
Azanza Sesé, Maider
Pereira Varela, Juanan

June 20, 2026

Abstract

This bachelor thesis addresses the challenge of managing and generating technical documentation when information is distributed across multiple sources and the writing process becomes time-consuming, repetitive, and difficult to maintain. To address this problem, we developed a web-based platform that integrates a FastAPI backend, a React frontend, PostgreSQL with pgvector, and an AI agent layer based on Mattin AI, an agent-configuration application developed by LKS Next, the company where the project was carried out. The platform enables users to create workspaces, upload reference files, generate Markdown documents, and edit content through editable blocks enriched with semantic relationships and contextual assistance. By combining document management, AI-assisted generation, and structured editing, the system supports a more efficient and consistent documentation workflow. As a result, this thesis delivers a modular and extensible document-authoring tool that improves the generation, editing, and maintenance of technical documentation.

Sustainable Development Goals

This project contributes primarily to **Sustainable Development Goal 9: Industry, Innovation and Infrastructure**[1]. The project presented in this work is a multi-agent software system for AI-assisted generation and block-level editing of professional documents. Its main contribution is not in the physical infrastructure domain, but in the development of digital infrastructure that can improve how organisations create, maintain, and refine technical and professional documentation.

Many professional environments still rely on document workflows that are slow, repetitive, and difficult to keep consistent when several sections must be updated at the same time. The system developed addresses that problem through a modular architecture that combines specialised agents, semantic relationships between document blocks, and controlled editing pipelines. In practical terms, this helps reduce manual effort, improves traceability of changes, and supports more efficient knowledge-intensive work. From the perspective of SDG 9, this is relevant because innovation is not limited to creating new products: it also includes designing better digital processes that make organisations more productive, reliable, and adaptable.

The thesis also has a secondary connection with **SDG 4: Quality Education**[2]. Academic, technical, and institutional documents are essential for teaching, learning, and knowledge transfer. A system that helps users generate structured content, revise specific sections, and preserve consistency across a document can make high-quality written material easier to produce and update. This is especially valuable in contexts where users need support to transform complex ideas into clear and well-organised documentation.

Even though this project is a prototype and not a final industrial product, it explores a realistic application of artificial intelligence to augment human work rather than replace it. For that reason, its strongest and most defensible contribution is to SDG 9, by advancing innovative digital tooling for professional document engineering.

[1] https://sdgs.un.org/goals/goal9
[2] https://sdgs.un.org/goals/goal4

Contents

Contents v

List of Figures viii

List of Tables ix

1 Introduction 1

2 Background 3
- 2.1 LLMs and Generative AI . . . 3
- 2.2 Retrieval-Augmented Generation and Contextual Processing . . . 4
- 2.3 MCP Servers . . . 4
- 2.4 AI-Assisted Document Authoring Systems . . . 4
  - 2.4.1 Evolution of AI in Text Generation and Editing . . . 5
  - 2.4.2 Structured Documentation and Lightweight Markups . . . 5
  - 2.4.3 Current Solutions and Approaches . . . 6
  - 2.4.4 Key Limitations in Existing Workflows . . . 6
- 2.5 Mattin AI . . . 7

3 Planning 9
- 3.1 Scope and objectives . . . 9
- 3.2 Project life cycle and stages . . . 10
- 3.3 Task breakdown . . . 11
  - 3.3.1 First iteration: scope, requirements, and architecture . . . 11
  - 3.3.2 Second iteration: base system implementation . . . 12
  - 3.3.3 Third iteration: completion of user and document management . . . 12
  - 3.3.4 Fourth iteration: export, finalisation, and deployment . . . 13
- 3.4 Task timelines and milestones . . . 13
  - 3.4.1 Task Dependencies . . . 13
  - 3.4.2 Gantt Chart and Milestones . . . 15
- 3.5 Time management . . . 17
- 3.6 Risk management . . . 17
- 3.7 Stakeholders Management . . . 20
- 3.8 Information and communication management . . . 21

3.8.1 Information systems . . . 21
3.8.2 Communication systems . . . 21
3.8.3 Management and control plan . . . 22

4 Requirements Analysis **23**
4.1 Main Requirements . . . 23
4.1.1 Functional Requirements . . . 23
4.1.2 Non-Functional Requirements . . . 24
4.1.3 Non-Measurable Constraints . . . 25

5 System Design and Technologies **27**
5.1 Overall Architecture . . . 27
5.2 Conceptual Data Model . . . 29
5.3 Agent and Pipeline Design . . . 31
5.3.1 Agent Roles . . . 31
5.3.2 Pipeline Design . . . 32
5.4 Technology Stack . . . 35
5.4.1 Backend . . . 35
5.4.2 Database . . . 36
5.4.3 Frontend . . . 36
5.4.4 AI Integration . . . 37
5.4.5 GitHub Copilot and Custom Agents . . . 38

6 System Implementation **39**
6.1 User Authentication . . . 39
6.2 Workspace and File Management . . . 40
6.3 Document Editing . . . 42
6.4 Agent and Pipeline Implementation . . . 43
6.4.1 Generation Pipeline Implementation . . . 44
6.4.2 Editing Pipeline Implementation . . . 44
6.5 MCP Callback Channel . . . 47

7 System Evaluation **49**
7.1 Functional Testing Approach . . . 49
7.1.1 End-to-End Test Automation . . . 50
7.1.2 Continuous Integration with GitHub Actions . . . 50
7.2 Non-Functional Testing . . . 53
7.2.1 Security and Confidentiality . . . 53
7.2.2 Performance . . . 53
7.2.3 Reliability . . . 54
7.2.4 Maintainability and Deployment . . . 55
7.3 Benefits of the Adopted Strategy . . . 57
7.4 Limitations and Future Work . . . 57
7.5 Conclusion . . . 58

8 Monitoring and Control **59**
8.1 Scope Management 59
8.2 Time Tracking 60
8.2.1 Deadlines 60
8.2.2 Task Dedication 60

9 Conclusions **63**
9.1 Overall Conclusions 63
9.2 Project Objectives Review 64
9.3 Lessons Learned 65
9.4 Future Work 65

Appendices **67**

A Declaration of use of generative AI **67**

Bibliography **69**

List of Figures

2.1 Screenshot of Mattin AI, once we create an app. 7

3.1 Task Breakdown Diagram 11
3.2 Task Dependencies Diagram 14
3.3 Gantt Chart 16
3.4 Task Estimate 18

5.1 Architecture Diagram 28
5.2 Domain Model UML 30
5.3 Document Generation Pipeline Scheme 33
5.4 Editing Pipeline Scheme 35

6.1 Login Page 40
6.2 Main page 41
6.3 Workspace Page 42
6.4 Document Editing Page 43
6.5 Agents Configured in Mattin AI 44

7.1 Execution of an automated Playwright functional test. 50
7.2 GitHub Actions end-to-end testing pipeline executed on a self-hosted runner with local infrastructure dependencies. 52
7.3 LangSmith tracing 54
7.4 Data Structure Configuration in Mattin AI 54
7.5 Agent Configuration in Mattin AI 55

8.1 Task Dedication Table 61

List of Tables

2.1 Categorisation of current AI-assisted authoring solutions. 6

3.1 Main project milestones. 15

5.1 Agents and responsibilities 32

6.1 MCP tools exposed by the backend. 48

8.1 Estimated and Real Dates of Milestones. 60

CHAPTER 1

Introduction

Recent advances in generative Artificial Intelligence, and especially in Large Language Models (LLMs), have opened new possibilities for supporting writing and editing tasks [1, 2]. In document authoring, however, generating fluent text is not enough: documents must also preserve structure, incorporate reference material, remain consistent across sections, and support iterative revision [3].

Current AI tools are useful, but they often fall into two extremes. Traditional editors provide formatting and manual editing features, yet offer limited intelligent assistance; generic conversational assistants can generate text, but usually operate outside the real document environment and struggle to maintain coherence across large structured documents [3]. This project addresses that gap through a web-based system for AI-assisted generation and block-level editing of documents. The system combines specialised agents, block-based document modeling, and MCP-enabled tooling so that users can generate an initial document from contextual material and then refine it in a controlled, iterative workflow.

Meanwhile, LKS Next[1] has developed an application named Mattin AI[2], allowing users to create and develop AI agents, as well as add-ons such as mcp-servers, skills and other features to adapt these agents. With the help of this application, the project aims to develop the aforementioned system, as we go about day-to-day work in a company such as LKS Next.

The system developed, together with its full source code, is publicly available in the project repository at Github: https://github.com/TelmoGoiko/TFG

The remainder of this report is organised as follows. Chapter 2 introduces the technical and conceptual background required to understand the project, including LLMs, retrieval-augmented generation, MCP, current AI-assisted authoring systems, and the Mattin AI platform. Chapter 3 presents the planning of the project, including the work breakdown, timeline,

[1] https://www.lksnext.com/es/
[2] https://mattin.ai

and milestones. Chapter 4 defines the functional and non-functional requirements that guided the development.

Chapter 5 describes the overall system design, architecture, and selected technologies. Chapter 6 explains the implementation of the different modules and agent pipelines. Chapter 7 presents the testing and validation strategy adopted for the system. Chapter 8 reviews the deviations, scope changes, and project monitoring activities carried out throughout development. Finally, Chapter 9 summarises the main conclusions of the work, the lessons learned, and the future lines of development.

CHAPTER 2

Background

Having set out the aim of this project in the introduction, this chapter will describe the general concepts required to understand this document. To this end, we will first provide a broad overview of what agents based on large language models (LLMs) are, how they work, and how they interact with external tools. We will also introduce the Model Context Protocol as a communication standard between these components.

Building on this, we will describe the current state of the art in AI-assisted document authoring systems, as this is the environment in which we will be developing the project.

Finally, we will explain how the Mattin AI app from LKS Next works; this will be a key component of the project, providing us with the storage systems and agents with access to RAG that we require.

2.1 LLMs and Generative AI

Large Language Models are neural network models designed to process and generate natural language through sequences of tokens [1]. Instead of operating directly on words or sentences, these models transform text into token-based numerical representations, which allow them to capture syntactic patterns and semantic relationships learned during training on large-scale text corpora. This training process enables LLMs to perform a wide range of language-related tasks, including text generation, summarisation, question answering, and content refinement.

A key concept in the operation of LLMs is the use of vector representations, often referred to as embeddings, which encode linguistic meaning in a mathematical form that can be processed by the model. In practical applications, many LLM-based systems rely on instruct-tuned models, which have been further adapted to follow user instructions more effectively [2]. This makes them particularly suitable for interactive systems intended to support writing, editing, and other forms of knowledge work.

2.2 Retrieval-Augmented Generation and Contextual Processing

Although LLMs are capable of generating coherent and useful text, their responses are limited by the knowledge encoded during training and by the amount of context provided in the prompt. Retrieval-Augmented Generation (RAG) addresses this limitation by combining language generation with the retrieval of relevant external information [4]. In this approach, the system first identifies and retrieves content related to the user request, and then incorporates that information into the model input before generating a response.

This method is especially valuable in document-oriented environments, where the quality of the generated output depends on the availability of accurate and context-specific information. In practice, RAG-based systems often rely on embeddings and similarity search mechanisms to retrieve the most relevant document fragments. As a result, they are better suited for tasks that require grounded generation, contextual assistance, and the reuse of existing knowledge sources.

2.3 MCP Servers

Model Context Protocol (MCP)[1] servers provide a structured way for agents to interact with external tools, resources, and execution environments. Rather than limiting an agent to text generation alone, MCP servers allow it to access additional capabilities such as reading files, querying services, or invoking specific operations required to complete a task. This turns the agent into a more practical computational component, capable of combining reasoning with execution.

From a software engineering perspective, MCP servers are relevant because they introduce a standardised interface between intelligent agents and external system functionality. This improves modularity and interoperability, while also making it easier to extend agent behaviour without tightly coupling it to a specific implementation. In agent-based systems, this approach is particularly useful when task completion depends not only on language understanding, but also on controlled access to external context and operational tools.

2.4 AI-Assisted Document Authoring Systems

The integration of Artificial Intelligence (AI) into document authoring has shifted the paradigm from traditional word processing to collaborative content generation. This section examines the current landscape of AI-assisted writing tools, focusing on structured environments and lightweight formatting workflows, while outlining their core capabilities and inherent limitations.

[1] https://modelcontextprotocol.io/

2.4.1 Evolution of AI in Text Generation and Editing

Early document authoring assistants were primarily limited to rule-based grammar checking and basic autocomplete features. However, the advent of Large Language Models (LLMs) has enabled semantic understanding and context-aware generation [1, 2]. Modern systems assist authors across three main pillars:

- **Content Generation:** Drafting initial sections, brainstorming outlines, and expanding bullet points into cohesive prose based on user prompts.
- **Content Editing and Refinement:** Adjusting tone, summarising lengthy paragraphs, translating text, and ensuring stylistic consistency throughout the document.
- **Structural Organisation:** Automatically generating tables of contents, cross-referencing sections, and formatting bibliographies.

2.4.2 Structured Documentation and Lightweight Markups

In writing, efficiency is highly dependent on separation of concerns and separating content from presentation. This is where lightweight markup languages, particularly Markdown[2], have become the standard for modern documentation pipelines (often referred to as "Documentation as Code").

AI-assisted systems designed for these environments must operate differently than those built for rich-text editors (like Microsoft Word or Google Docs[3]). They need to:

- Respect and manipulate syntax trees (e.g., Markdown Abstract Syntax Trees) to ensure that AI-generated content does not break document formatting or metadata headers (YAML front matter).
- Integrate seamlessly into text editors and Integrated Development Environments (IDEs) via extensions (e.g., VS Code[4] extensions, Obsidian[5] plugins).
- Support modular documentation, where large documents are split into smaller files that the AI must analyze contextually to maintain coherence.

[2] https://daringfireball.net/projects/markdown/
[3] https://workspace.google.com/products/docs/
[4] https://code.visualstudio.com/
[5] https://obsidian.md/

2.4.3 Current Solutions and Approaches

State-of-the-art solutions can be categorised into three main approaches, that are explained in the following Table 2.1:

Approach	Description	Examples
General-Purpose Assistants	Web-based LLM interfaces used alongside traditional editors.	ChatGPT, Claude, Gemini
Embedded AI Editors	Rich-text or Markdown editors with native AI capabilities.	MD Editor, Cursor, Nimbalyst
IDE Extensions	Plugins that integrate directly into the developer's writing workflow.	GitHub Copilot, Llama Coder, Custom VS Code extensions

Table 2.1: Categorisation of current AI-assisted authoring solutions.

Representative examples of these approaches include ChatGPT[6], Claude[7], Gemini[8], Cursor[9], and GitHub Copilot[10].

2.4.4 Key Limitations in Existing Workflows

Despite their capabilities, current AI-assisted authoring systems suffer from several critical drawbacks that motivate the development of new solutions:

- **Context Fragmentation:** Many tools struggle to maintain long-range context across large, multi-file documents, leading to contradictions or repetitive phrasing [3].
- **Lack of Domain Specificity:** Generic LLMs often fail to strictly adhere to specific technical constraints or organisational style guides without extensive prompt engineering [2].
- **Syntax Disruptions:** Non-specialised AI models frequently generate malformed Markdown code, breaking rendering engines or destroying document structure.
- **User Agency and Over-automation:** Finding the balance between "human-in-the-loop" refinement and automated generation remains an open challenge [5], as fully automated text often lacks the nuanced depth required for technical documentation.

By analysing these limitations, we will endeavour to minimise their impact through the project by proposing a system that addresses and eliminates them.

[6] https://chatgpt.com/
[7] https://claude.ai/
[8] https://gemini.google.com/
[9] https://www.cursor.com/
[10] https://github.com/features/copilot

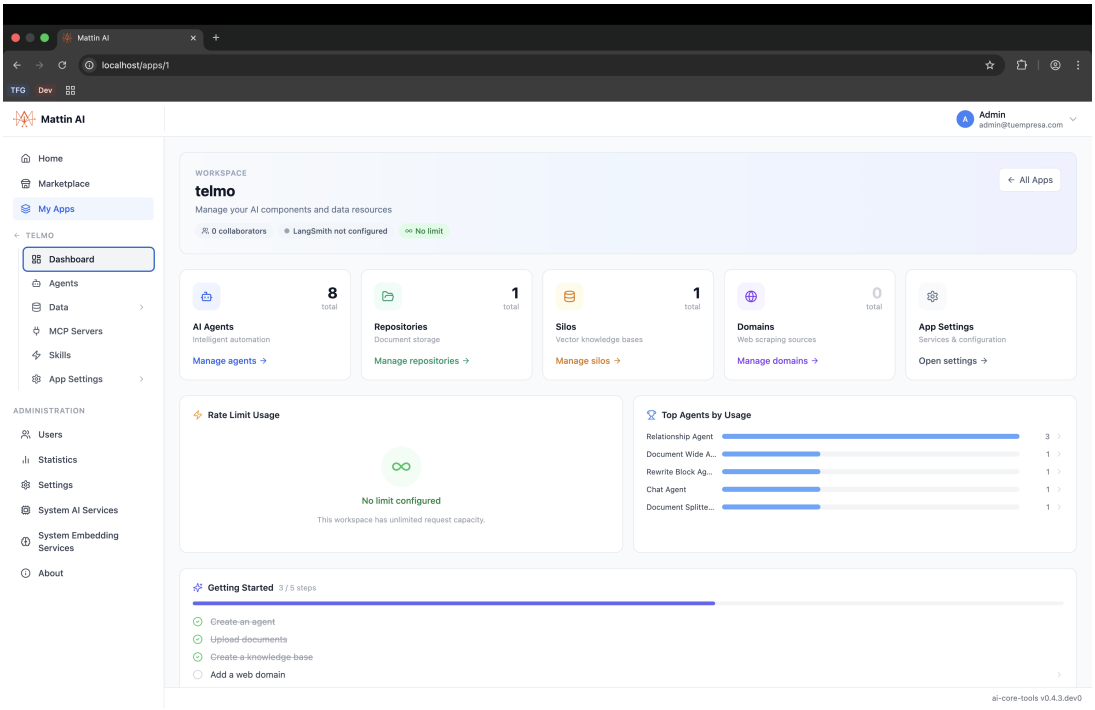


Figure 2.1: Screenshot of Mattin AI, once we create an app.

2.5 Mattin AI

Mattin AI is a platform for building AI applications and agents based on a modular architecture. Its aim is not merely to connect to a language model, but to provide a comprehensive environment for configuring agents, linking them to documentary knowledge, exposing them via APIs, and managing all of this within isolated workspaces. In practice, Mattin AI allows the user to create custom agents, choose which model they use, add memory to them, define structured outputs and connect them to a knowledge base so they can respond with context.

Conceptually, Mattin AI is organised around an *App* or *workspace*. Within each *App*, *AI services*, *agents* and *knowledge resources* are configured. An *agent* is the main entity: it has a system prompt, an associated language model and, optionally, access to tools and a RAG system. Two important concepts that we already use come into play here: *repositories* and *silos*. The *repository* is the document organisation layer, where files are uploaded and structured. The *silo* is the vector knowledge base where the embedded representations of those documents are stored so they can be retrieved semantically. Put simply: the *repository* stores and organises the files; the *silo* stores their representation so that the *agent* can search for them by meaning, not just by name or literal text. We can see a screenshot of the Mattin AI app in Figure 2.1.

The general process works as follows. First, an *app* is created and the necessary services are configured, typically a language model and an embedding model. Next, a *repository* is

created, and automatically a *silo* linked to it is generated, which acts as a repository for vectorised knowledge. Then, usually, documents are uploaded to the *repository*. When these documents are ingested, the system processes them, divides them into fragments, and generates embeddings for each fragment. These embeddings are stored in the *silo*. Later, when the user makes a query to an *agent* connected to that *silo*, Mattin AI transforms the question into a vector representation, searches the *silo* for the most relevant fragments and passes them to the language model as context. The *agent* then generates a response based on the uploaded documentation.

From a functional perspective, Mattin AI combines three layers. The first is the generation layer, based on LLMs. The second is the document retrieval layer, based on *repositories* and *silos*. The third is the orchestration layer, which manages *agents*, conversational memory, permissions, APIs and configuration. This makes Mattin AI particularly useful for scenarios where it is not enough simply to 'ask a model', but where a reproducible system is needed to create specialised agents with access to their own knowledge.

CHAPTER 3

Planning

This chapter presents the project planning based on the updated planning document provided. It describes the scope of the system, the main objectives, the iterative organisation of the work, the estimated effort, the most relevant risks, and the mechanisms defined for information and communication management.

3.1 Scope and objectives

The scope of the project is to develop an application that helps users generate and edit documents in a professional context, such as offers, contracts, or similar work-related documents, with the support of personalised LLM agents defined in LKS Next's Mattin AI. More specifically, the aim is to build a minimal but functional system where, given a repository containing one or more reference documents and a new user prompt, an agent can generate a new document with similar characteristics.

The generated document will be divided into multiple Markdown units, referred to as *blocks*, to make editing easier and more precise. In addition, each block will be associated with a chatbot supported by reviewing agents and an MCP server that provides auxiliary functions. This design allows the user to refine specific parts of the document without having to request a complete regeneration of the whole result. As the project is based on innovative and evolving technologies, its scope is not considered fully closed from the start and may be refined throughout development.

The main functional objectives of the system are the following:

- **Provide a platform** for generating and editing documentation within a specific workspace, using the available agents and the documentation accessible in the repository.

- **Offer a responsive chat** with an agent so that the user can edit the information in each block, while also being able to manually add or delete information and edit tables or data.
- **Automate the update of related blocks** so that each modification is correctly reflected in connected sections, maintaining the coherence and synchronisation of the document.
- **Ensure the connection and concordance** between the developed application and the information stored in Mattin AI repositories and agents.

In addition to these functional objectives, two transversal goals are also pursued. First, the application should be **easily extensible**, so that new features can be incorporated in the future through a well-structured implementation. Second, the project should follow a methodology that closely resembles a **real enterprise development process**.

3.2 Project life cycle and stages

An incremental and iterative life cycle is proposed for the project, with iterations lasting approximately two weeks. This approach allows the development process to adapt to emerging requirements and to the knowledge gained while working with Mattin AI, agents, and the MCP server [6].

At a general level, four main iterations are defined:

1. **First iteration:** planning, requirements analysis (explained in Chapter 4), scope definition, and architecture design. During this stage, Mattin AI and its codebase are studied in order to understand how persistence and information management must be handled.
2. **Second iteration:** implementation of the base system, including the most technically demanding features, such as document generation, block editing, agent integration, and MCP-supported interactions.
3. **Third iteration:** completion of the system by incorporating user management and document management, together with the persistence and synchronisation of documents with Mattin AI.
4. **Fourth iteration:** finalisation of the system, including export features, interface refinement, validation and deployment tasks, and completion of the academic work related to the thesis report and the defense.

This life cycle allows the objectives of the project to be achieved progressively and in a structured manner. At the same time, it admits some overlap between non-dependent tasks, and it also allows parts of the thesis to be written before implementation is fully completed, especially those related to planning and requirements.

3.3 Task breakdown

The task breakdown structure has been defined according to the adopted iterative-incremental model. Figure 3.1 shows a diagram of the work packages, and the following subsections summarise them and their role in each stage of the project.

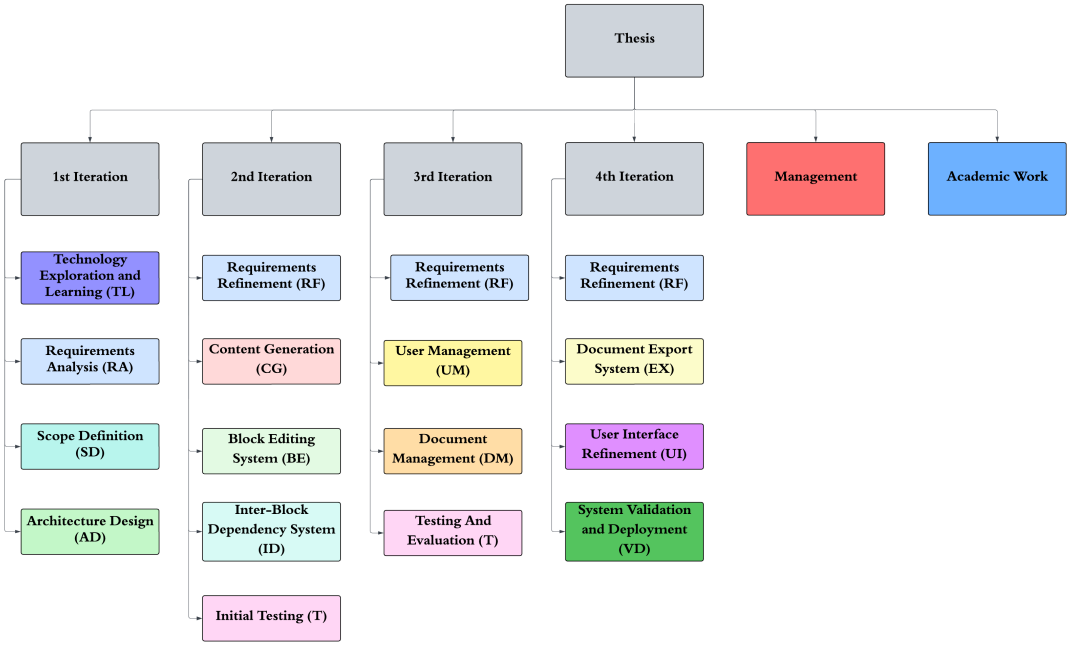


Figure 3.1: Task Breakdown Diagram

3.3.1 First iteration: scope, requirements, and architecture

The first iteration establishes the conceptual and technical foundations of the project:

- **Technology Exploration and Learning (TL):** familiarisation with Mattin AI, LLM-based systems, agent architectures, and MCP server concepts required for the project development.
- **Requirements Analysis (RA):** identification and documentation of functional and non-functional requirements, including content generation, document management, and block editing, as well as the analysis of user needs in AI-assisted document generation.
- **Scope Definition (SD):** identification of the key functionalities of the system and study of similar AI-based documentation tools in order to define the main modules of the application.

- **Architecture Design (AD):** definition of the web architecture, the integration with Mattin AI, the agent system for generation and review, and the functions to be implemented in the MCP server.

3.3.2 Second iteration: base system implementation

The second iteration focuses on the implementation of the technical core of the application:

- **Requirements Refinement (RF):** revision of the initial requirements after the technical exploration of Mattin AI and the first implementation efforts. This includes refining the JSON structure, validation rules, fallback strategies, expected agent behaviour, and MCP interaction constraints.
- **Content Generation (CG):** user prompt handling, context retrieval from the repository, integration with the generation agent, structured JSON generation, conversion into Markdown blocks, initial document assembly, and integration of the agent with MCP for context access and structural validation.
- **Block Editing System (BE):** manual editing of Markdown blocks, block creation and deletion, block state management, contextual chatbot integration, transfer of block context to the agent, update of blocks based on the agent's response, and block-based change history management.
- **Inter-block Dependency System (ID):** detection of semantic relationships between blocks, automatic consistency updates between related sections, and use of MCP tools to identify affected blocks and propagate changes.
- **Initial Testing (T):** unit tests, integration tests, expected-versus-actual validation of agent outputs, MCP call testing, and end-to-end verification of the agent–MCP–block workflow.

3.3.3 Third iteration: completion of user and document management

Once the base system is operational, the third iteration focuses on the application features required for practical usage and persistence:

- **Requirements Refinement (RF):** validation and adjustment of block-editing requirements, consistency rules, agent-assisted editing behaviour, user control over automatic updates, and user–system interaction patterns based on actual usage.
- **User Management (UM):** user registration, authentication, session handling, access control, and mapping between users, workspaces, and agents.
- **Document Management (DM):** workspace creation, integration with Mattin AI as a storage system, document upload and deletion, document visualisation, and persistence of blocks and per-block chat interaction history.

- **Testing and Evaluation (T):** validation of editing features, consistency testing, quality assessment of agent output, and testing of complete workflows from user interaction to agent response and block updating.

3.3.4 Fourth iteration: export, finalisation, and deployment

The fourth iteration gathers the final work needed to prepare the system for delivery:

- **Requirements Refinement (RF):** final validation of all functional requirements, adjustment of export requirements, refinement of non-functional constraints such as performance, usability, and deployment readiness, validation of AI-generated content quality, definition of the final review workflow, and prioritisation of the remaining improvements.
- **Document Export System (EX):** assembly of the final document from validated Markdown blocks, consistency verification after editing and review, export to Markdown format, and export to PDF format.
- **User Interface Refinement (UI):** interface optimisation, workspace navigation refinement, improvement of block visualisation, and incorporation of visual feedback for edits, updates, and AI suggestions.
- **System Validation and Deployment (VD):** end-to-end validation of the complete workflow, performance evaluation, deployment in a web environment, and verification of concurrent multi-user scenarios.

In parallel with the four iterations, two additional lines of work are maintained throughout the project: **management**, including planning and regular monitoring meetings, and **academic work**, including the writing of the thesis report and the preparation of the defense.

3.4 Task timelines and milestones

3.4.1 Task Dependencies

The dependencies among the project tasks require a structured and partially sequential execution, while still allowing some overlap between independent activities. Figure 3.2 illustrates these relationships.

The project starts with the learning and exploration of technologies, followed by the planning phase, where the scope, objectives, and time constraints are established. This phase supports the first iteration, in which requirements analysis, scope definition, and architecture design are carried out in a strongly interdependent way.

During the second iteration, the project begins with a refinement of requirements and then moves to the implementation of content generation. Once generated blocks exist, the block editing system can be implemented, followed by the inter-block dependency system,

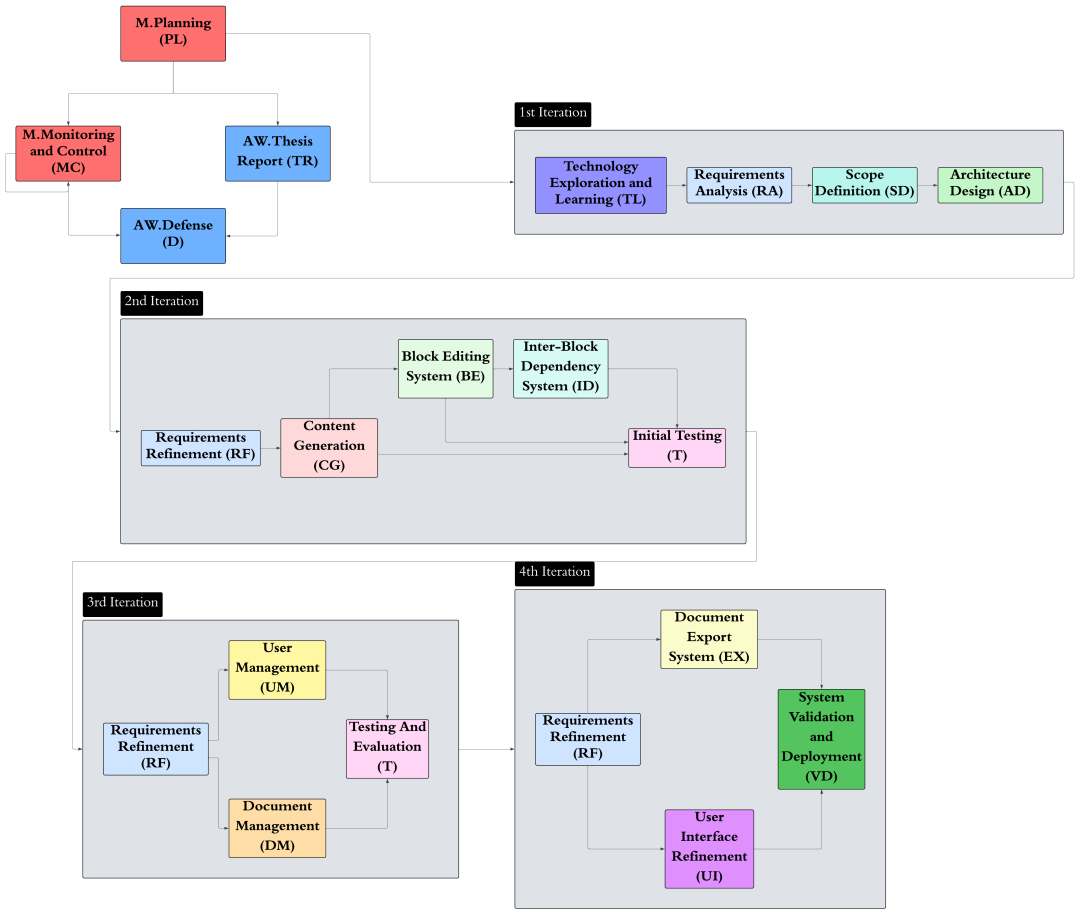


Figure 3.2: Task Dependencies Diagram

which depends on editable content being available. Initial testing is then performed to validate the entire pipeline, including agents, MCP tools, and block transformations.

The third iteration again starts with requirements refinement and continues with user management and document management. These tasks enable authentication, access control, persistence, and synchronisation with Mattin AI. Testing at this stage focuses on editing workflows, agent interactions, consistency management, and the correct behaviour of the persisted system.

Finally, the fourth iteration refines the remaining requirements and implements export, interface improvements, and system validation and deployment. In parallel with the technical development, monitoring and control activities continue, and the thesis report is drafted progressively so that planning- and requirement-related sections can be completed early.

3.4.2 Gantt Chart and Milestones

Figure 3.3 shows the Gantt chart, which provides an approximate overview of the time allocation for each work package throughout the project. The diamonds indicate the key milestones detailed in Table 3.1.

Milestone	Date
Start of the project	24/03/2026
End of 1st milestone	10/04/2026
End of 2nd milestone	01/05/2026
End of 3rd milestone	22/05/2026
End of 4th milestone	05/06/2026
End of thesis and project	14/06/2026
Defense	T.B.D.

Table 3.1: Main project milestones.

A relevant planning decision is that the implementation phase is expected to end on June 5, 2026. After that, two full weeks are reserved for writing the report until June 14, 2026, leaving a seven-day contingency buffer before the submission deadline.

The project begins with the planning work package, followed by the initiation of project monitoring and control alongside the first iteration. From the second iteration onward, the system development packages are focused, extending through to the fourth iteration. In parallel, work begins on drafting the thesis, as sufficient material is available for the first chapters. Finally, two weeks are dedicated exclusively to the thesis, followed by preparation for defense.

This schedule allows efforts to be focused according to the priorities of each phase: initially planning and defining the scope, implementing the system in the intermediate phase, and academic work at the end, whilst maintaining continuous drafting of the thesis throughout the development.

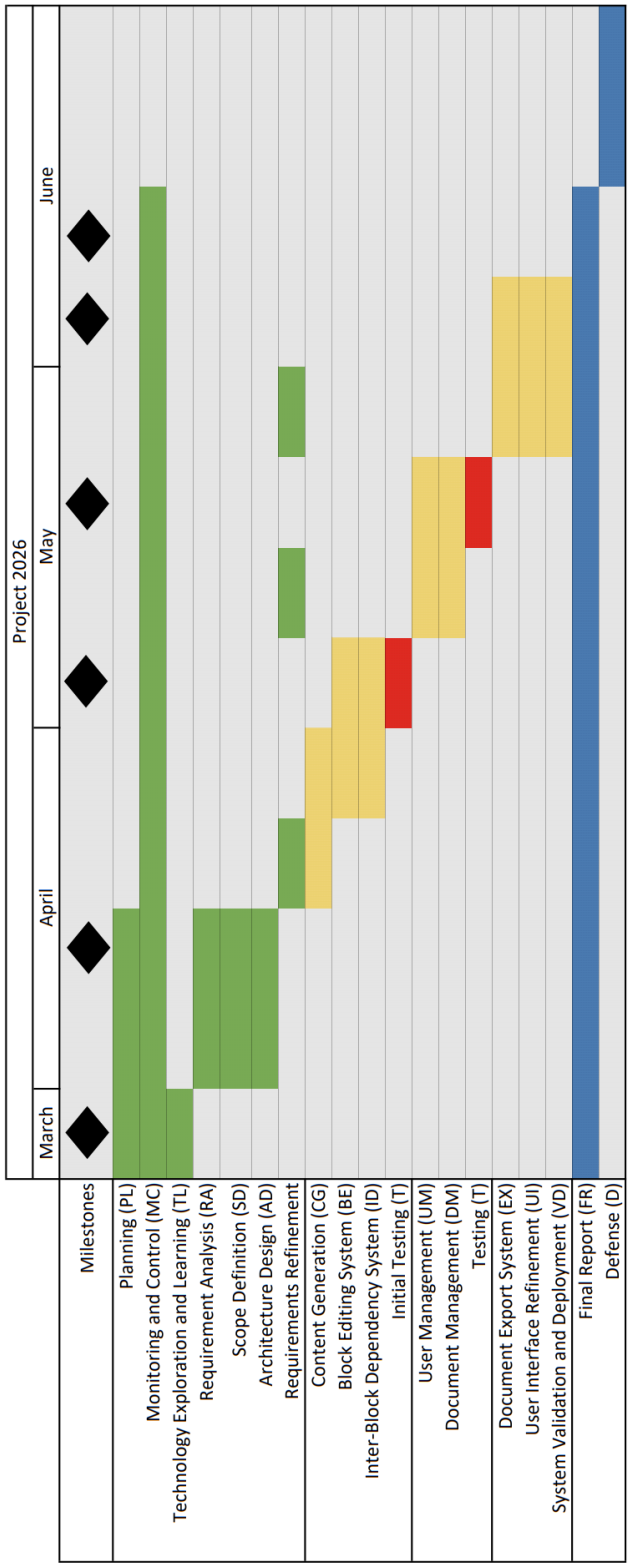


Figure 3.3: Gantt Chart

3.5 Time management

The time allocated to the project has been distributed according to the defined scope and the technical complexity of each stage. A larger proportion of hours is assigned to the second iteration because it includes the implementation of editable content blocks, consistency management, MCP server usage, and the integration of the agents. The third iteration also receives a significant allocation because it adds user functionalities and completes the connection with Mattin AI for document creation and persistence. The fourth iteration requires fewer hours, since it concentrates on export, interface refinements, deployment, and final validation over already implemented modules.

Figure 3.4 summarises the estimated workload.

Overall, the project is estimated at 385 hours, including technical development, project management, and academic work.

3.6 Risk management

Because the project depends on AI agents, MCP-supported interactions, and integration with external systems, a number of significant risks have been identified:

- **R1 – Inappropriate planning and management of the project:** Project management such as planning or control may be inadequate due to the lack of experience on the author's part and to be dealing with a project that is mainly exploratory.
  - **Impact**: Medium
  - **Mitigation**: Use an iterative life cycle, at the end of the cycle we have a potential product. The scope of the project can be discussed later. Frequent control meetings to detect early deviations.
  - **Contingency Plan**: Call a meeting, discuss scope with the company. Replan accordingly.
- **R2 – Integration issues with Mattin AI:** The project depends on the correct interaction with Mattin AI for document persistence and retrieval. Limited documentation, compatibility issues, or unexpected behaviour may delay development.
  - **Impact**: High
  - **Mitigation**: Conduct early exploration of Mattin AI during the first iteration and develop small integration prototypes before full implementation.
  - **Contingency Plan**: Implement temporary mock repositories for testing while resolving integration problems.
- **R3 – Inconsistent JSON outputs from AI agents:** AI agents may generate malformed or incomplete JSON structures, affecting Markdown block generation.

Iteration	Task	Estimated Hours
1st Iteration	Technology Exploration and Learning (TL)	10
	Requirement Analysis (RA)	20
	Scope Definition (SD)	10
	Architecture Design (AD)	20
	Total 1st Iteration	60
2nd Iteration	Content Generation (CG)	40
	Block Editing System (BE)	35
	Inter-Block Dependency System (ID)	40
	Initial Testing (T)	10
	Total 2nd Iteration	125
3rd Iteration	User Management (UM)	10
	Document Management (DM)	30
	Testing (T)	10
	Total 3rd Iteration	50
4th Iteration	Document Export System (EX)	15
	User Interface Refinement (UI)	10
	System Validation and Deployment (VD)	15
	Total 4th Iteration	40
Management	Planning (PL)	20
	Monitoring and Control (MC)	10
	Total Management	30
Academic Work	Final Report (FR)	70
	Defense (D)	10
	Total Academic Work	80
TOTAL	-	385

Figure 3.4: Task Estimate

  - **Impact**: High
  - **Mitigation**: Define strict structured-output schemas for each agent within Mattin AI, validate every agent response against these schemas before persisting it, and design prompts that explicitly constrain the expected output format to reduce malformed responses.
  - **Contingency Plan**: When validation fails, automatically retry the agent call with a corrective prompt; if the error persists, fall back to a manual correction step so the user can fix or discard the affected output.

- **R4 – Errors in MCP tool execution:** Failures in MCP server functions (e.g., incorrect tool usage, unexpected outputs, or communication issues) may affect agent behaviour and system reliability.
  - **Impact**: High
  - **Mitigation**: Design robust tool interfaces, validate inputs and outputs, and implement error handling for MCP calls.
  - **Contingency Plan**: Introduce fallback behaviours or bypass MCP-dependent features when errors occur.

- **R5 – Difficulty maintaining consistency between blocks:** Automatic synchronisation between related blocks, especially when triggered by agents, may introduce inconsistencies or unintended changes.
  - **Impact**: High
  - **Mitigation**: Design dependency detection carefully and validate updates incrementally with controlled rules.
  - **Contingency Plan**: Disable automatic updates temporarily and notify users of related blocks requiring manual review.

- **R6 – Performance issues due to agent and MCP interactions:** Frequent agent executions, context processing, and MCP calls may increase response times and affect user experience.
  - **Impact**: Medium
  - **Mitigation**: Optimise prompts, reduce unnecessary context, and cache repeated operations where possible.
  - **Contingency Plan**: Prioritise critical workflows and limit non-essential agent interactions.

- **R7 – Unpredictable agent behaviour in editing workflows:** Agents may produce unexpected modifications when editing blocks, affecting document quality or user trust.
  - **Impact**: Medium

  - **Mitigation**: Constrain agent behaviour through structured prompts and clearly defined scopes of action.
  - **Contingency Plan**: Allow users to review, reject, or manually adjust agent-generated changes.

- **R8 – Scope expansion:** New desired features may appear during development.
  - **Impact**: Medium
  - **Mitigation**: Maintain clear iteration goals and prioritise core requirements.
  - **Contingency Plan**: Move lower-priority improvements to future work.

- **R9 – Data security and confidentiality risks:** The system processes commercially sensitive information, such as pricing data, contracts, and internal strategies. The use of external LLM services may introduce risks related to data exposure, unauthorised access, or non-compliance with data protection regulations such as GDPR [7].
  - **Impact**: Medium
  - **Mitigation**:
    * Implement strict access control mechanisms based on user roles and authentication
    * Apply data anonymisation or minimisation techniques when interacting with LLMs
    * Use secure environments and, where possible, private or enterprise-grade LLM APIs
    * Ensure compliance with applicable data protection regulations (e.g., GDPR)
  - **Contingency Plan**:
    * Restrict or disable features that rely on sensitive data processing if risks cannot be adequately mitigated
    * Introduce manual review processes for outputs involving critical or confidential information

3.7 Stakeholders Management

The development of the project involves the following stakeholders:

- **Author:** student and executor of the project itself, whose graduation depends on the project's success.
- **Advisors:** two academic advisors and one company advisor, whose supervision of the project forms part of their professional responsibilities.
- **LKS Next:** organisation financing part of the development, and that will benefit from the content generated.

- **UPV-EHU:** academic institution responsible for overseeing the project, which contributes to its educational and applied research mission.

3.8 Information and communication management

Information and communication management is a key factor in ensuring traceability, coordination, and supervision throughout the project life cycle.

3.8.1 Information systems

Different platforms are used depending on the nature of the information handled:

- **GitHub**[1]: the main hosting platform for the project code and the primary version control mechanism.
- **Overleaf**[2]: the platform used for collaborative writing of the thesis together with the academic supervisor.
- **Google Drive**[3]: storage for supporting documentation and auxiliary materials created during the project.
- **Microsoft Tools**[4]: LKS Next provides a Microsoft account to keep track of the work done and communicate with other members of the organisation using **Outlook, Teams, and OneDrive**.

These systems make it easier to supervise the project, track changes, and recover previous versions if errors or redesign decisions occur.

3.8.2 Communication systems

Communication between the student, the company supervisor, and the academic supervisors relies on several channels:

- **E-mail:** the main channel for asynchronous communication and for sharing files and links.
- **Online meetings:** Microsoft Teams[5] is used for remote meetings and short communication exchanges through chat.

[1] https://github.com/
[2] https://www.overleaf.com/
[3] https://workspace.google.com/products/drive/
[4] https://www.microsoft.com/microsoft-365
[5] https://www.microsoft.com/microsoft-teams/

- **Face-to-face meetings:** in-person meetings can take place either at the company or at the faculty whenever the participants are available.

Overall, this communication and information management strategy is intended to ensure regular follow-up, good coordination among all parties, and an orderly handling of the materials generated throughout the project.

3.8.3 Management and control plan

Project monitoring and control will be supported through a regular coordination plan involving all the main stakeholders: the student, the company supervisor, and the academic supervisors. A follow-up meeting is planned every two weeks throughout the development period in order to review progress, detect deviations, and agree on the next short-term objectives.

These meetings are intended to provide continuous visibility over the state of the project and to ensure that both the academic and professional perspectives remain aligned. During each session, the work completed since the previous meeting will be reviewed, current risks and blockers will be discussed, and any necessary adjustments to scope, priorities, or deadlines will be decided.

In addition to progress tracking, this management and control plan is meant to support decision-making and reduce the impact of uncertainty inherent to a project that depends on evolving AI technologies, agent behaviour, and external integrations. Keeping a fixed biweekly cadence also helps anticipate problems early and ensures that all parties remain informed about the evolution of the system and the associated thesis work.

CHAPTER 4

Requirements Analysis

Once the scope has been presented in Chapter 3, and the background has been analysed in Chapter 2, it is now appropriate to define the set of requirements that must be satisfied in order to complete the project successfully. The contents presented in this chapter have been discussed and agreed upon with the relevant stakeholders.

4.1 Main Requirements

The project must achieve the following characteristics in order to meet the objectives previously proposed:

4.1.1 Functional Requirements

The functional requirements are organised into a set of core system modules intended to ensure that the application fulfills its scope as an AI-assisted document generation and block-based editing platform.

4.1.1.1 User and Workspace Management

- **Authentication and role-based access control.** The system shall provide secure user authentication together with a role-based access control (RBAC) mechanism. Each user shall be associated with one or more workspaces according to their permissions, thereby ensuring that sensitive commercial information remains accessible only to authorised actors.
- **Workspace isolation.** The system shall allow users to manage multiple projects independently through separate workspaces. Each workspace shall maintain its own repository of reference documents and its own set of assigned agents, thus preserving the contextual and operational independence of each project.

4.1.1.2 Document Generation Driven by Agents

- **Contextual generation through retrieval-augmented generation.** Given a user prompt and a repository of reference documents, the system shall implement an agent-led workflow capable of generating a new document that preserves the stylistic, structural, and contextual characteristics of the provided references [4].
- **Structured output transformation.** The generation agent shall produce its output in a structured JSON format, which the system shall automatically parse and transform into individual Markdown blocks that can be edited independently.
- **MCP tool integration.** The agents shall make use of an MCP server in order to access auxiliary capabilities, such as structural validation, consistency checking, and contextual retrieval from the Mattin AI repository.

4.1.1.3 Block Editing and Synchronisation

- **Per-block conversational assistance.** Each Markdown block shall be associated with a specific chatbot. Users shall be able to request localised modifications, for example, rewriting a table or refining a paragraph, without affecting the remainder of the document.
- **Inter-block dependency management.** The system shall detect semantic relationships between blocks. Whenever a modification in one block may affect the content or validity of another block, the system shall notify the user and, where appropriate, suggest or trigger an update through the MCP server.
- **Manual overrides.** Users shall retain full editorial control over the document. Consequently, the system shall allow direct manual editing of text, the addition or deletion of blocks, and direct modification of tables without requiring AI intervention.

4.1.1.4 Persistence and Mattin AI Integration

- **State persistence.** The system shall persist the current state of all document blocks together with the complete interaction history associated with each block-specific agent within the Mattin AI infrastructure.
- **Export mechanism.** The application shall provide a mechanism for assembling all validated blocks into a single coherent document, which shall be exportable at least in Markdown and PDF formats.

4.1.2 Non-Functional Requirements

The non-functional requirements define the expected quality attributes of the system and, where applicable, include measurable objectives that may be used during validation.

4.1.2.1 Security and Confidentiality

- **Data isolation.** The system shall guarantee strict separation of data across workspaces.
  - **Metric:** cross-workspace data access.
  - **Measurement method:** code review and penetration testing focused on detecting unauthorised API access between different user workspaces.
  - **Objective:** zero leakage of sensitive commercial information, including prices, strategies, and subcontracting data, between users or workspaces.

4.1.2.2 Performance

- **Agent response time.**
  - **Metric:** elapsed time between a block-edit request and the corresponding agent response.
  - **Objective:** an average response time below 10 seconds for minor block refinements, excluding the network latency associated with external LLM calls.
- **Consistency synchronisation.**
  - **Metric:** time required to detect and propagate changes between dependent blocks.
  - **Objective:** background detection and user notification in under 10 seconds.

4.1.2.3 Reliability

- **JSON robustness.**
  - **Metric:** rate of malformed AI responses leading to application failures.
  - **Objective:** all malformed JSON outputs shall be intercepted by validation schemas and shall trigger either an automatic retry or a manual correction prompt.

4.1.2.4 Maintainability and Deployment

- **Containerisation.** The complete system, including the application, the MCP server, and the database connectors, shall be containerised in order to ensure consistent behaviour across development and production environments.

4.1.3 Non-Measurable Constraints

In addition to the requirements listed above, the project is subject to the following constraints, which are relevant from an organisational and operational perspective, even if they are not directly measurable through quantitative metrics.

- **Documentation.** Complete technical documentation covering the MCP server architecture and the agent prompts shall be provided in order to facilitate future maintenance and knowledge transfer.
- **Cost control.** The system should incorporate a cost-conscious prompting strategy aimed at minimising token consumption when interacting with external large language models.

CHAPTER 5

System Design and Technologies

Having outlined the project proposal and its management, this chapter sets out both the internal and external components of the system, as well as the technologies used in its development.

5.1 Overall Architecture

The project consists of four main layers: the frontend, the backend, the database, and external integration with Mattin AI. To develop the app, we have organised the project into a directory structure that manages the frontend, backend, database migrations, and related connections. This architecture is illustrated in Figure 5.1.

- **Frontend:** We can find all the files that make up the website, with the main code located in the src folder, and the following subfolders:
  - **pages/:** One file per screen in the application, covering the full user journey from workspace management to document generation and block editing.
  - **components/:** Reusable interface pieces shared across multiple pages, grouped into layout elements, editor-specific components (block list, chat panel, relationship viewer, impact warnings), and generic utility components.
  - **services/:** All communication with the backend is centralised here. These files are also responsible for translating the API's data format into the format the rest of the frontend uses.
- **Backend:** Manages the entire implementation and connections between the frontend, the database and external services within its own folder. We can also find the main code in the app folder with the following subfolders:

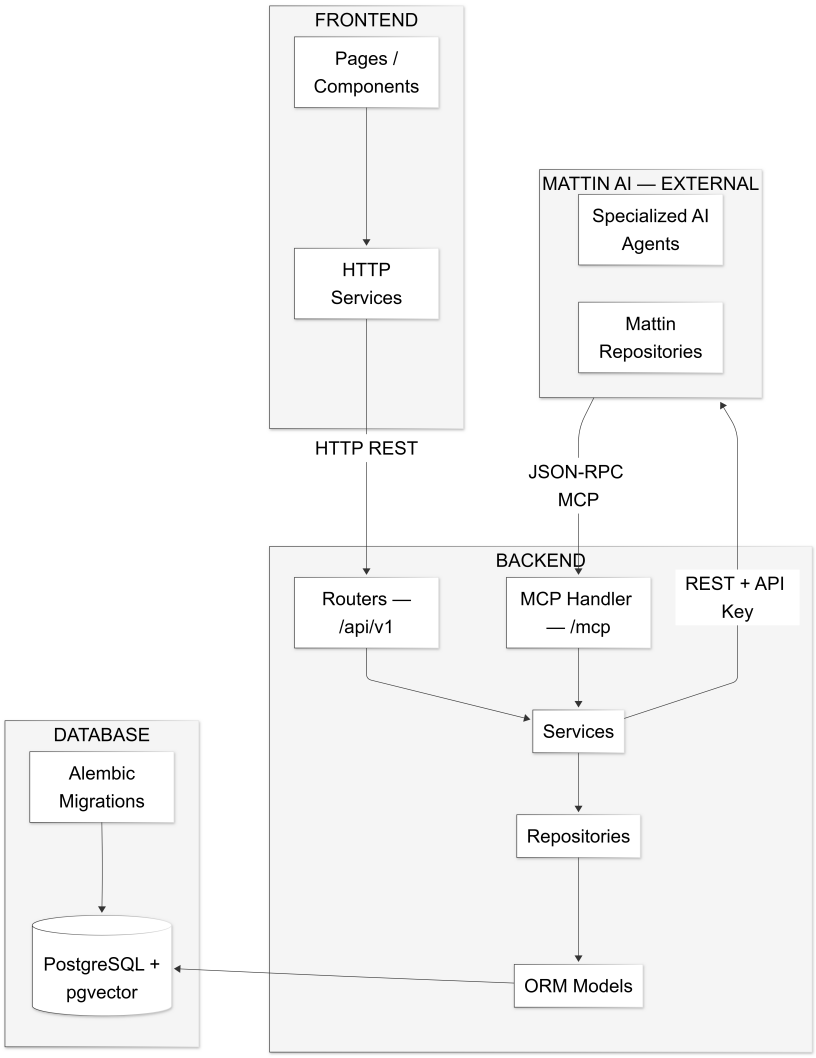


Figure 5.1: Architecture Diagram

- **routers/:** The entry point for every HTTP request. Each file groups a set of related endpoints and delegates all logic to the service layer.
- **services/:** Where all business logic lives. This includes a central orchestrator for workspace operations and a set of specialised services for AI-driven features such as impact analysis, relationship detection, and document generation. Agent-specific logic is further isolated in a dedicated agents/ subfolder.
- **repositories/:** All database queries are isolated here, away from business logic.
- **models/:** One file per database table, defining its structure and columns.
- **integrations/:** The only place allowed to communicate with the Mattin AI API.

  - **mcp_handler/:** Implements the tools that Mattin AI agents can call back on the backend during an active generation session.
  - **core/:** Application-wide configuration, read from environment variables at startup.

- **Database:** The database stores all persistent data that is not stored in Mattin AI's database. We thought about relying completely on the Mattin AI Database but, in the end, we concluded that using a local database was preferable to manage everything related to workspaces runs and blocks, as we needed specific models, and making these changes in Mattin AI would involve making pull requests and reviews of the code, relying on the workers in Mattin AI and perhaps delaying the work.

- **Mattin AI (External Integration):** Mattin AI works as an external AI platform that the backend communicates with in two directions.

  - **Outbound:** When the user triggers an AI or upload action (generate a document, chat with a block, rewrite content, check for impact, upload documents for knowledge), the backend calls the corresponding Mattin AI agent via its REST API. There is a dedicated agent for each type of operation, identified by a numeric ID in the application configuration.
  - **Inbound:** During document edition and generation, Mattin AI agents call back into the backend through a dedicated /mcp endpoint, using it to read and write blocks incrementally as the document is being written.

5.2 Conceptual Data Model

The system's data flow follows a clear hierarchical structure rooted in the User entity. A user owns one or more Workspaces, which act as the primary organisational unit of the application. Each workspace can hold reference material in the form of WorkspaceFiles, binary uploads such as PDFs or images which can be provided as context to the AI generation engine.

When the user triggers a generation session, a WorkspaceRun is created under the workspace, capturing the original prompt and its lifecycle status. The run produces an ordered collection of Blocks, each representing a piece of the generated file, with its title, type, summary, and full content. Once blocks exist, the system analyzes their dependencies and establishes BlockRelationships, directed links that describe how one block relates to another (e.g., one file importing or extending another).

As blocks are edited, the AI detects downstream effects and generates ImpactSuggestionRecords, proposals for updating other blocks that may be affected by the change. Each block also maintains its own conversational history through ChatMessages, allowing the user to iteratively refine the content through a per-block AI chat.

All entities, their attributes, and the relationships described above are visually reflected in Figure 5.2.

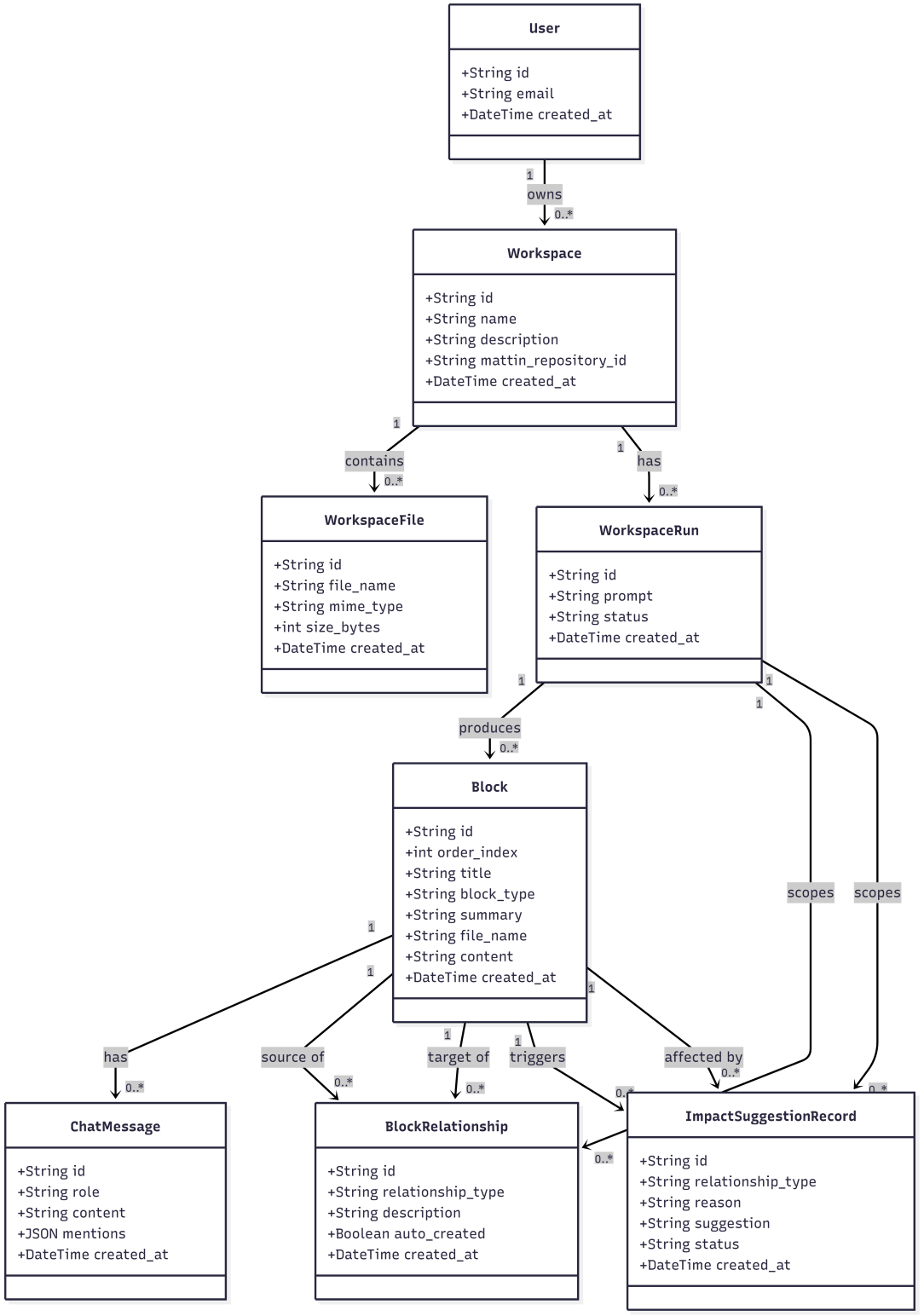


Figure 5.2: Domain Model UML

5.3 Agent and Pipeline Design

The AI-assisted generation and editing functionality of the system is structured around a set of specialised agents, coordinated through two pipelines: a *generation pipeline*, which produces a new document from a user prompt and reference material, and an *editing pipeline*, which supports the block-level refinement of that document.

5.3.1 Agent Roles

Eight agents are designed, each with a single, well-scoped responsibility. This single-responsibility design keeps prompts short and focused, reduces hallucination risk [8], and allows each agent to be tuned or replaced independently. The proposed agents are shown in Table 5.1:

How these agents are configured within the Mattin AI platform, including the underlying language model and structured-output settings for each one, is described in Section 6.4.

Agent	Responsibility
Document Writer	Generates a full Markdown document from a user prompt and optional file references. Returns a structured JSON object `{title, summary, markdown}`.
Document Splitter	Receives the writer's output and divides it into a list of independently editable blocks `{title, block_type, summary, markdown}`.
Block Chat	Conversational assistant scoped to a single block. Proposes edits to the current block, lists cross-block rewrites that the backend executes automatically and can invoke MCP tools to create or delete blocks when the user requests structural changes.
Block Rewrite	Applies a single targeted instruction to one block and returns the full rewritten Markdown. Invoked by the backend in parallel for each cross-block rewrite produced by the Chat agent.
Block Impact	Detects consistency implications when a block is saved. Given the diff of the changed block, it analyses all blocks that semantically depend on it and generates an actionable suggestion for each.
Block Relationship	Scans all blocks after generation and infers semantic relationships (*references*, *depends_on*, *contradicts*, *extends*). Returns a JSON array that the backend stores in the database.
Document Wide	Applies a document-wide edit across all blocks simultaneously. Invoked when the Chat agent detects that the user's request targets the whole document (e.g. "change all occurrences of X").
Visual Generator	Generates embeddable visual assets (charts via QuickChart) on demand. Called as a tool when the user requests a graph or image. Returns visual_markdown, description where visual_markdown is a ready-to-embed Markdown string.

Table 5.1: Agents and responsibilities

5.3.2 Pipeline Design

5.3.2.1 Generation Pipeline

Document generation is designed as a three-stage pipeline (Figure 5.3): a write stage, a split stage, and a relationship-detection stage.

- **Write stage:** The Document Writer agent receives the user prompt together with a list of reference document titles and, when available, file references that Mattin AI can embed as context. It returns a single coherent Markdown document.
- **Split stage:** The resulting Markdown is passed to the Document Splitter agent, which

identifies natural structural boundaries (chapters, sections, or paragraphs depending on document length) and returns an ordered list of blocks. An index block is automatically generated and persisted together with the rest.

- **Relationship detection stage:** Once all blocks are persisted, the Block Relationship agent receives a condensed representation of every block (ID, title, type, summary, and a short content preview). It returns a set of directed semantic relationships (*references*, *depends_on*, *contradicts*, *extends*), consistent with the `BlockRelationship` entity introduced in Section 5.2. These relationships are later used by the Chat and Impact agents to provide contextually aware suggestions.

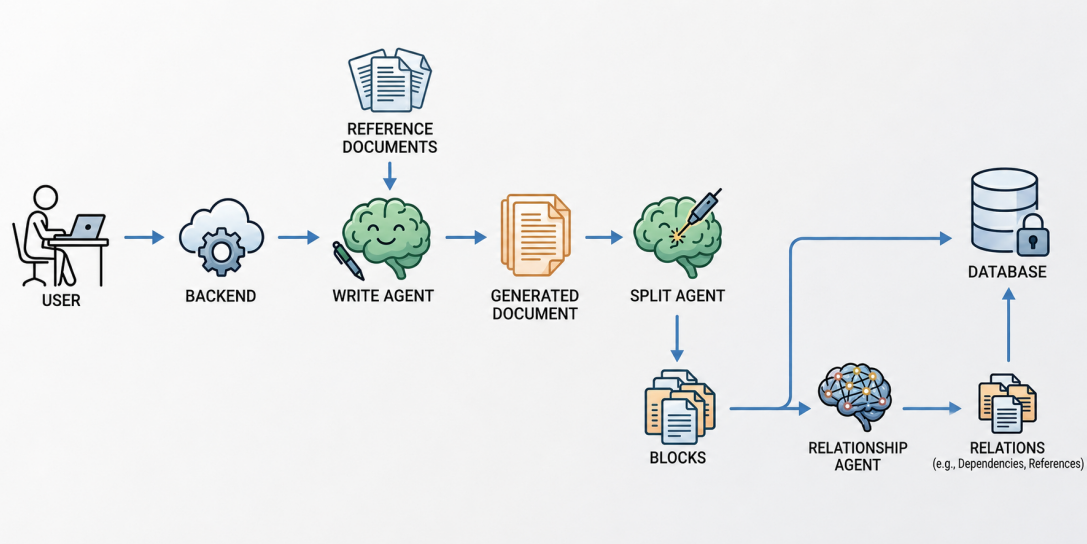


Figure 5.3: Document Generation Pipeline Scheme

5.3.2.2 Editing Pipeline

Once a document exists, users interact with individual blocks through a chat interface. The editing pipeline is designed around three agents working in coordination, plus two supporting agents invoked under specific conditions.

Before any agent is invoked, the system performs *scope detection*: it determines whether the user's request targets a single block or the whole document. Document-wide requests are routed to the **Document Wide** agent, which receives the full content of every block and returns a batch of rewrites applied across the document. Otherwise, the request is handled at block scope by the **Block Chat** agent.

For block-scoped requests, the Block Chat agent receives:

- The current block's content and type.
- The full document outline (all block titles and summaries).
- The content of semantically related blocks (derived from stored relationships).
- The user's message and any selected text snippet.
- The workspace and run identifiers (so it can invoke MCP tools if needed).

The agent responds with an assistant message, an optional updated version of the current block, and a list of cross-block rewrites, each one identifying another block that also needs changes together with a precise natural-language instruction. These cross-block rewrites are dispatched concurrently, invoking the **Block Rewrite agent** once per affected block, so that the Chat agent's own prompt stays small while edits still propagate across the document.

When the user's request involves generating a visual asset such as a chart, graph, or diagram, the Chat agent invokes the **Visual Generator** as a tool rather than producing the visual itself. The Visual Generator determines whether the request involves a quantitative comparison or a time series and produces a ready-to-embed chart, which is inserted directly into the block's updated content so that no extra round trip with the frontend is required. Generated charts are persisted locally so that the document does not remain dependent on the availability of the external charting service.

After a block is saved, the system asynchronously evaluates its impact on the rest of the document: it identifies the blocks linked to the changed one through an incoming *references* or *depends_on* relationship and invokes the **Block Impact** agent once per affected block. The resulting suggestions are stored as `ImpactSuggestionRecord` entities (Section 5.2) and surfaced to the user without blocking the save operation.

Figure 5.4 summarises this flow.

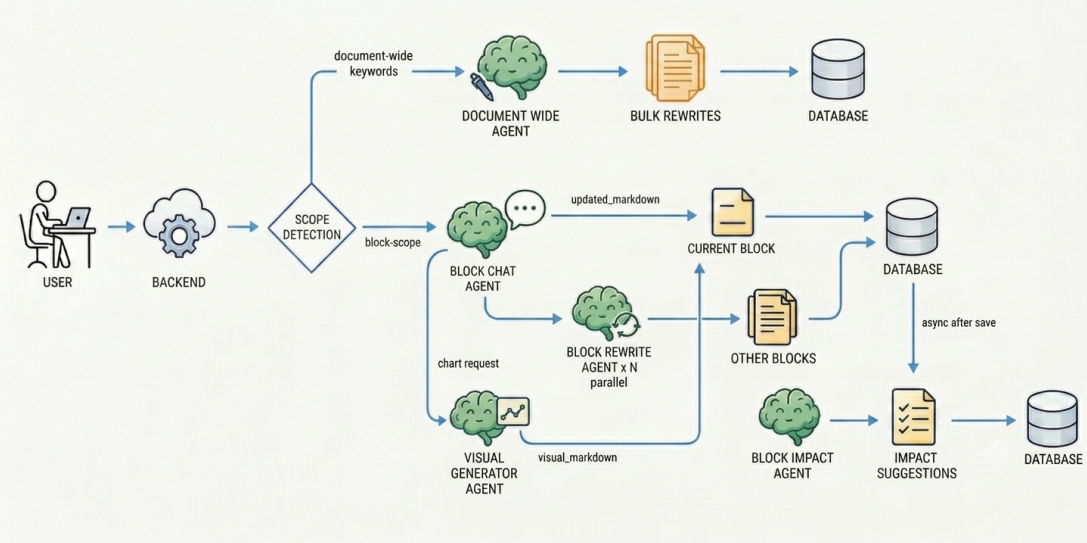


Figure 5.4: Editing Pipeline Scheme

5.4 Technology Stack

This section presents the technologies, frameworks, and tools that form the system's architecture, grouped by layer. For each one, its role within the project is described along with the rationale for its selection. It should be noted that, as it was unsure whether the code would be implemented in the Mattin AI application or not, the same type of technology was implemented in our app, so that it could be integrated if needed.

5.4.1 Backend

- **Python 3.11:** Python is the primary language for the server-side layer. Its extensive ecosystem for data processing and AI integration, combined with mature web frameworks, made it the natural choice for a project centred on document intelligence and LLM orchestration.
- **FastAPI**[1]: FastAPI is a modern, high-performance web framework for building REST APIs with Python. It leverages Python type hints to provide automatic request validation via Pydantic and generates interactive OpenAPI documentation out of the box. Its asynchronous-first design (built on Starlette and Uvicorn) allows the server to handle I/O-bound operations—such as calls to the external AI platform—efficiently without blocking.
- **SQLAlchemy 2.0**[2]: SQLAlchemy is used as the ORM (Object-Relational Mapper) to abstract database interactions. Version 2.0 introduces a fully typed, session-based API

[1] https://fastapi.tiangolo.com/
[2] https://www.sqlalchemy.org/

that integrates cleanly with FastAPI's dependency injection system. It manages all entity relationships—workspaces, runs, blocks, and their relationships—and generates the SQL queries sent to PostgreSQL.

- **Alembic**[3]: Alembic is the database migration tool associated with SQLAlchemy. It tracks schema changes through versioned migration scripts, enabling controlled, reproducible evolution of the database schema across environments. Every structural change to the data model is encoded as an Alembic revision, ensuring that the database can be brought to any historical state deterministically.

- **Pydantic v2 and pydantic-settings**[4]: Pydantic v2 is used throughout the backend for data validation and serialisation. All API request and response bodies are defined as Pydantic models, providing automatic type coercion and validation. `pydantic-settings` extends this to application configuration, loading environment variables into a typed `Settings` object at startup, which eliminates runtime configuration errors.

5.4.2 Database

- **PostgreSQL 16 with pgvector**[5,6]: PostgreSQL is the relational database used to persist all application data. The `pgvector` extension augments PostgreSQL with native support for storing and querying high-dimensional vector embeddings, enabling semantic similarity search directly within the database without the need for a separate vector store.

- **Docker and Docker Compose**[7,8]: The database is containerised using Docker, with the official `pgvector/pgvector:pg16` image. Docker Compose orchestrates the service, binding the container to a persistent volume and exposing it on a configurable port. This approach ensures maintaining a fully functional database throughout all the project's phases.

5.4.3 Frontend

- **React 19**[9]: React is the UI library used to build the single-page application. Its component model promotes reusability and a clear separation of concerns between UI elements.

[3] https://alembic.sqlalchemy.org/
[4] https://docs.pydantic.dev/
[5] https://www.postgresql.org/
[6] https://github.com/pgvector/pgvector
[7] https://www.docker.com/
[8] https://docs.docker.com/compose/
[9] https://react.dev/

- **React Router v7**[10]: React Router provides client-side routing, mapping URL paths to page components. It manages navigation between the workspace list, document editor, generation views, and authentication pages.

- **Vite**[11]: Vite is the build tool and development server for the frontend. It relies on native ES modules in the browser during development, delivering near-instant hot module replacement (HMR) regardless of project size. For production, it uses Rollup to produce optimised, tree-shaken bundles.

- **react-markdown and remark-gfm**[12,13]: The `react-markdown` library renders Markdown content directly as React components, and `remark-gfm` extends it with GitHub Flavoured Markdown support (tables, strikethrough, task lists). These libraries are central to the document viewer and block editor, where content is stored and displayed in Markdown format.

- **Playwright**[14]: Playwright is used for end-to-end testing of the frontend application as explained in Chapter 7.

5.4.4 AI Integration

- **Mattin AI**[15]: Mattin AI is an external AI orchestration platform responsible for hosting and executing all AI agents used in the project (document writer, document splitter, block chat, block impact analysis, block relationship detection, and block rewrite). The backend communicates with Mattin AI via a dedicated HTTP client (`MattinClient`), forwarding user requests and receiving structured JSON responses. Mattin AI also manages the underlying LLM calls and embedding generation through its own configured AI and embedding services (powered by OpenAI[16]).

- **Model Context Protocol (MCP)**[17]: The backend exposes a set of workspace tools—listing blocks, reading block content, creating and deleting blocks—through an MCP-compatible endpoint. This allows Mattin AI agents to call back into the application to read and modify document state during a generation run, effectively giving the AI agents grounded, real-time access to the workspace data. MCP decouples the agent logic from the application layer, enabling the AI to act as an autonomous collaborator rather than a simple request-response service.

[10] https://reactrouter.com/
[11] https://vite.dev/
[12] https://github.com/remarkjs/react-markdown
[13] https://github.com/remarkjs/remark-gfm
[14] https://playwright.dev/
[15] https://www.lksnext.com/en/technology/mattin/
[16] https://openai.com/
[17] https://modelcontextprotocol.io/

5.4.5 GitHub Copilot and Custom Agents

GitHub Copilot[18] is used throughout development as an AI-assisted coding tool integrated directly into VS Code[19]. Beyond generic code completion, the project defines a set of *custom agents* in the `.github/agents/` directory, each specialised in a specific layer of the codebase:

- **backend-expert:** Scoped to the `backend/` directory. It has deep knowledge of the FastAPI, SQLAlchemy 2.0, Pydantic v2, and Mattin AI integration conventions used in the project, and enforces the layered architecture (routers → services → repositories → models).
- **alembic-expert:** Specialised in creating and reviewing Alembic migration scripts, keeping the PostgreSQL schema in sync with the SQLAlchemy models while preserving the pgvector extension setup.
- **frontend-expert:** Scoped to the `frontend/` directory. It enforces React 19, React Router v7, Vite, and plain CSS conventions, and understands the service and normalisation layer that communicates with the backend.
- **git-github:** Handles Git and GitHub workflows: branching, commits, and pull requests using the GitHub CLI[20].

Each agent is defined as a Markdown file with a YAML front matter block that includes its name, description, and a set of *handoffs*, explicit transitions that allow one agent to delegate to another. For example, after **alembic-expert** creates a migration file, it can hand off to **git-github** to commit it, creating a lightweight multi-agent workflow directly within the IDE. This setup reduces context-switching during development and ensures that AI assistance is always grounded in the conventions of the specific layer being modified.

[18] https://github.com/features/copilot
[19] https://code.visualstudio.com/
[20] https://cli.github.com/

CHAPTER 6

System Implementation

As we already explained the architecture of the project and having in mind the shape of the application, this chapter describes the implementation of the system's core functionalities. As for the use of Mattin AI and its integration, we have cloned the project locally for our own use and connected it to our system via calls to the backend of our own Mattin AI instance through an API key.

6.1 User Authentication

Authentication is the first interaction any user has with the system. The login and registration screens are implemented in `LoginPage.jsx` (we can see it in Figure 6.1), which presents a form for email and password input and calls either `authService.loginWithEmail()` or `authService.registerWithEmail()` depending on the active tab.

On successful login, the backend returns the user object, containing `id`, `email`, and `created_at`. The frontend service stores it in `localStorage` under a configurable key, `tfg_auth_user` by default, and returns a normalised camelCase object to the application.

Route protection is implemented in `ProtectedRoute.jsx`. It reads `isAuthenticated` from `AuthContext` and, if the value is false, redirects to `login` while preserving the original location in router state so the user is returned to their intended destination after login.

It is worth noting that the initial project plan allocated significantly more effort to the authentication module, as the application was originally conceived with a production deployment in mind, thus implying proper session management, token refresh flows, and role-based access control. Following the scope change described in Chapter 8, we decided not to invest further in this area, as the system is intended as a functional prototype rather than a production-ready product. The current implementation is therefore deliberately minimal, covering only what is necessary to support the demo workflow.

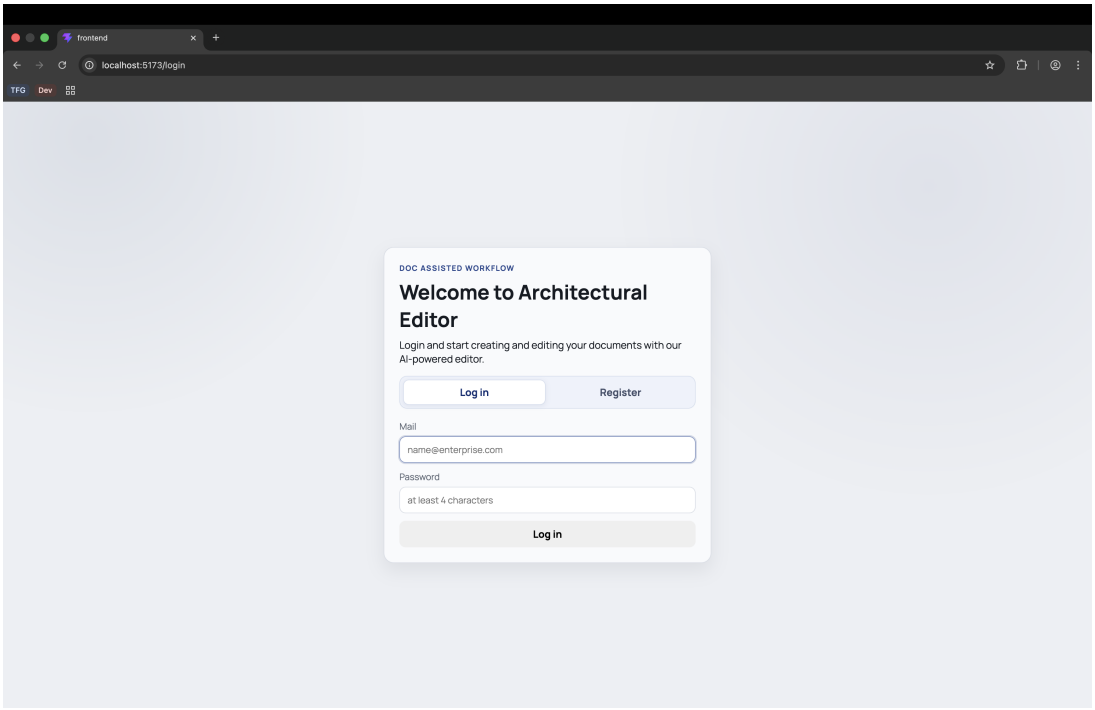


Figure 6.1: Login Page

6.2 Workspace and File Management

A workspace is the top-level container in the system, grouping the knowledge-base files and all generated documents that belong to a project. The workspace list screen, `MainPage.jsx` (Figure 6.2), and the workspace detail screen, `WorkspaceDetailPage.jsx` (Figure 6.3), together cover creation, browsing, file upload, and deletion.

When a user creates a workspace, the frontend calls the API with the workspace name and description alongside the `owner_id`. On the backend, `WorkspaceService.create_workspace()` follows an external-first commit strategy: it calls `MattinClient.create_repository()` to register a pgvector-backed repository in the Mattin AI platform first. Only if that call succeeds, the local `Workspace` commits the record to PostgreSQL, storing the returned `mattin_repository_id` for future synchronisation. This ordering ensures the local database never references a Mattin AI repository that does not exist.

This behaviour can be seen directly in the backend service implementation (Listing 6.1):

```
def create_workspace(self, owner_id: str, name: str, description:
    str) -> Workspace:
    if not name.strip():
        raise ValueError("Workspace name is required")

    try:
        created_repository = self.mattin_client.create_repository(
```

```
            name.strip())
    except MattinClientError as exc:
        raise ValueError(
            f"Could not create Mattin repository for workspace: {exc
                }"
        ) from exc

    repository_id = self._extract_mattin_identifier(
        created_repository)
    if repository_id is None:
        raise ValueError("Mattin repository creation did not return
            a repository id")

    model = Workspace(
        id=new_id(),
        owner_id=owner_id,
        name=name.strip(),
        description=description.strip(),
        mattin_repository_id=repository_id,
        created_at=datetime.now(UTC),
    )
    return self.repository.create_workspace(model)
```

Listing 6.1: External-first workspace creation

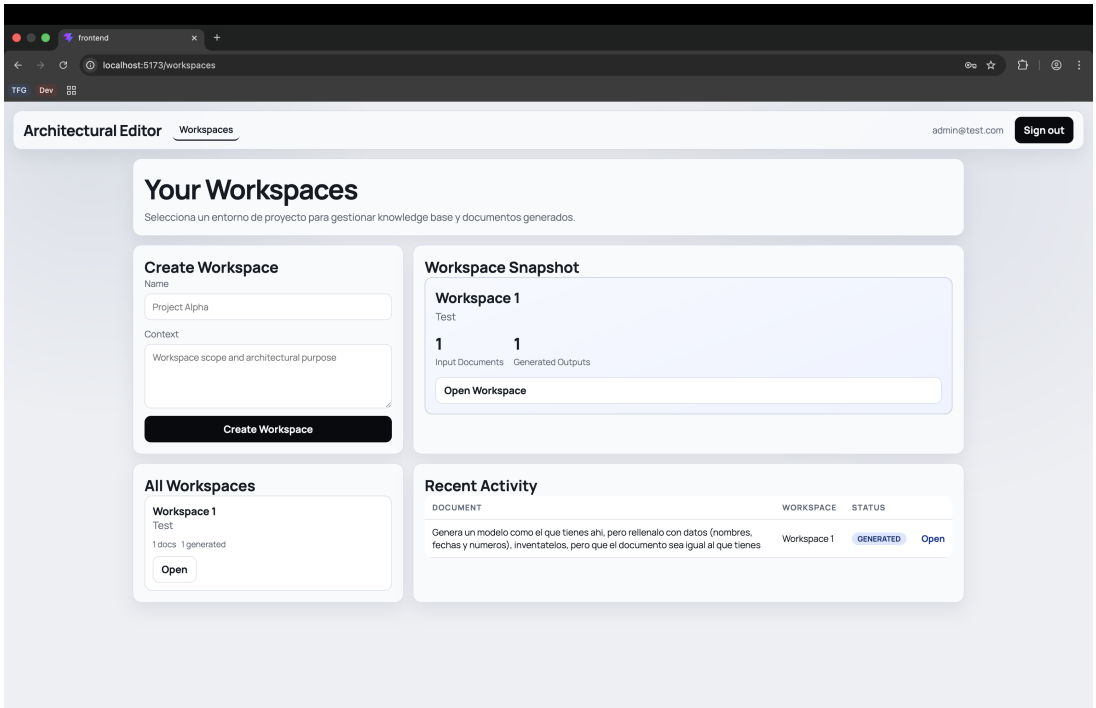


Figure 6.2: Main page

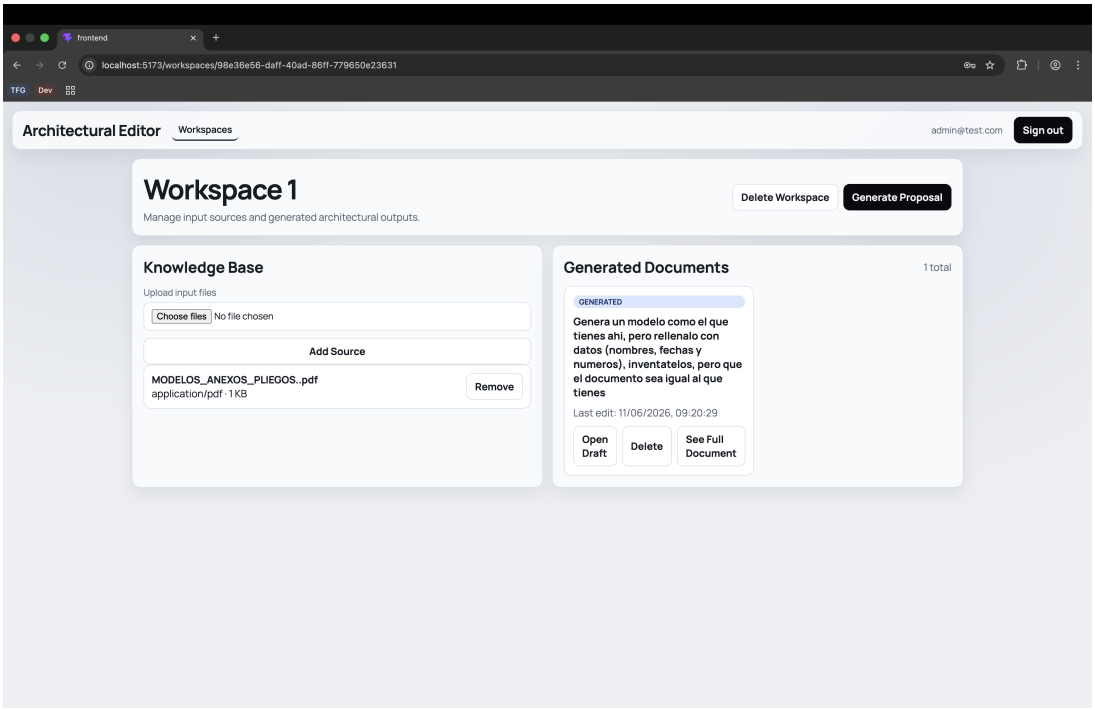


Figure 6.3: Workspace Page

This excerpt shows that the backend does not persist the workspace locally until the external repository has been created successfully. As a result, the local PostgreSQL state remains consistent with the Mattin AI platform even if the remote operation fails.

Every time the workspace list is fetched, `WorkspaceService._sync_workspaces_from_mattin()` runs before returning results. This method queries Mattin AI for the current list of repositories, compares it with the local records, and performs an upsert: new Mattin AI repositories are created locally, changed names are updated, and stale local records are removed. The same synchronisation pattern applies to file listings through `_sync_workspace_files_from_mattin()`. This bidirectional sync means the local database acts as a read cache that is always consistent with the Mattin state, rather than an independent store.

File uploads from the detail page are multipart API requests. The backend stores the file in Mattin AI via `MattinClient.upload_repository_file()`, using multipart form data with a 60-second timeout to accommodate large files, and simultaneously creates a `WorkspaceFile` record in PostgreSQL holding the `mattin_file_id` returned by Mattin AI.

6.3 Document Editing

Document editing is implemented as a block-based workflow in `BlockEditorPage.jsx`, where each chapter is shown in a split interface with a Markdown input pane and a live preview rendered with `ReactMarkdown+remark-gfm`. The editor keeps a per-block draft

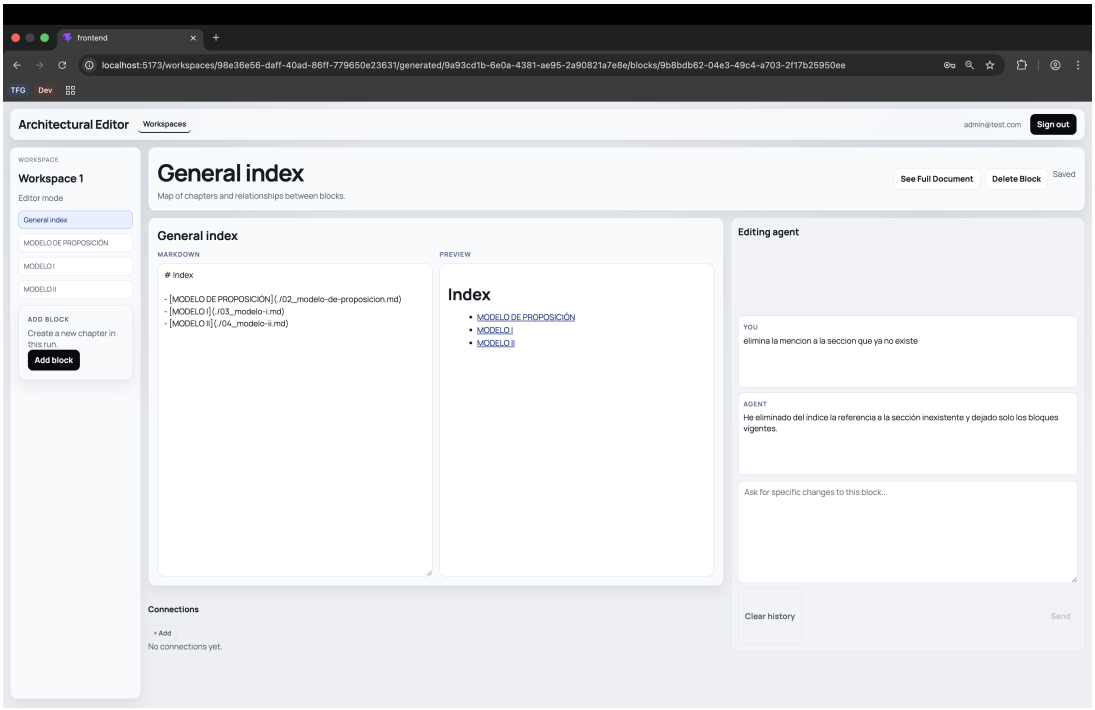


Figure 6.4: Document Editing Page

state and triggers an autosave cycle with a short debounce (1.5 s): before persisting, it calls `/check-impact` to precompute consistency suggestions, then updates the block through `PATCH /blocks/block_id`, refreshes the run, and reloads relationship metadata. In parallel, users can interact with the editing assistant through `ChatPanel.jsx`, which sends prompts to `/agent-chat` and stores conversation continuity via `conversation_id`; agent responses are presented as proposals (`proposed_content`) that the user can explicitly accept or reject, preserving human control over final edits. For document navigation and readability, `DocumentViewPage.jsx` renders all blocks in order and resolves internal Markdown links by mapping `file_name` to block routes, while external URLs open normally and unresolved references are marked as broken, providing immediate feedback about structural consistency during editing. We can see an example of the page in Figure 6.4.

6.4 Agent and Pipeline Implementation

The agent roles and pipelines designed in Section 5.3 are realised as eight agents configured directly in the Mattin AI platform, each one mapped to the responsibility described in Table 5.1. We can see them configured in Figure 6.5.

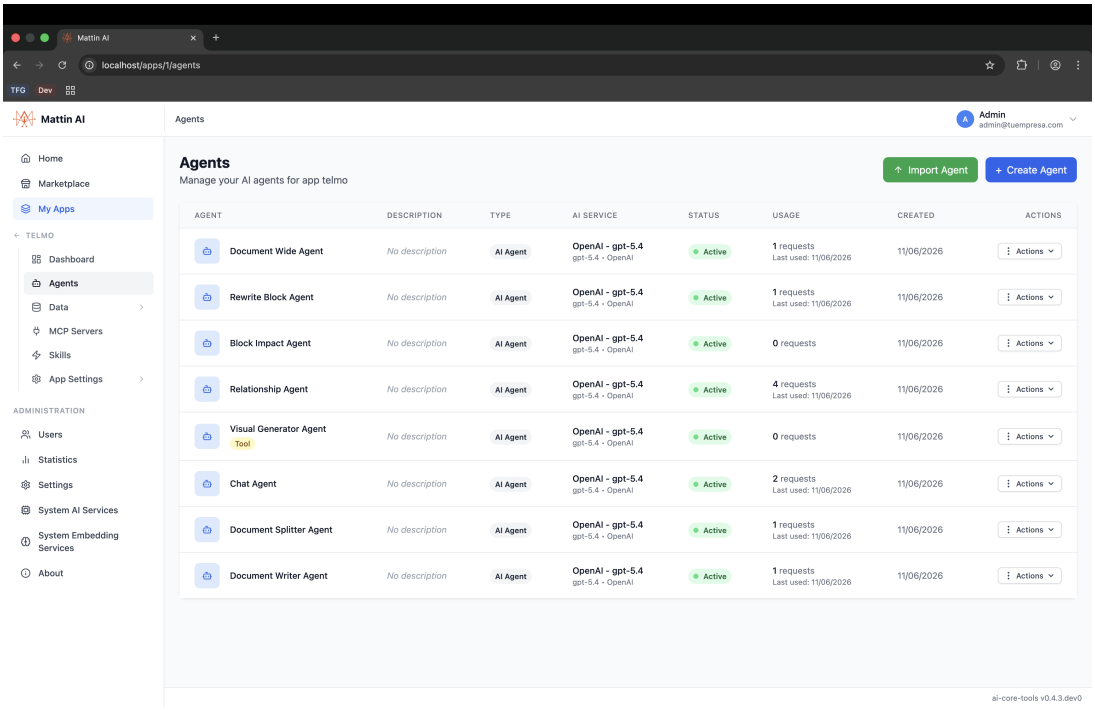


Figure 6.5: Agents Configured in Mattin AI

6.4.1 Generation Pipeline Implementation

The generation pipeline designed in Section 5.3.2.1 is orchestrated on the backend by `WorkspaceGenerationAgentService`, which sequences the calls to the Document Writer, Document Splitter, and Block Relationship agents, auto-generates the index block, and persists the resulting blocks and relationships once the Block Relationship agent's JSON array has been validated.

6.4.2 Editing Pipeline Implementation

- **Scope detection:** The routing between block-scoped and document-wide requests, described in Section 5.3.2.2, is implemented explicitly in the backend rather than delegated to the agent. The Chat service inspects the user's message against a fixed set of trigger keywords, such as "entire document", "all blocks", or "globally". If a document-wide intent is detected, the request is routed to the Document Wide agent, which returns a batch of rewrites that the backend applies sequentially in the same request cycle. This is shown in Listing 6.2:

```
    _DOCUMENT_WIDE_KEYWORDS: frozenset[str] = frozenset([
        "entire document", "whole document", "document-wide", "
            all blocks",
        "throughout the document", "across all blocks", "all
            occurrences",
```

```
        "globally", "every occurrence",
    ])

    def _is_document_wide_request(self, user_message: str) ->
        bool:
        lower = user_message.strip().lower()
        if any(kw in lower for kw in _DOCUMENT_WIDE_KEYWORDS):
            return True
        return False
```

Listing 6.2: Document-wide scope detection in the editing pipeline

- **Cross-block rewrites:** For block-scoped requests, the Block Chat agent responds with a structured JSON payload containing `assistant_message`, `updated_markdown` (nullable), and `cross_block_rewrites`, a list of other block IDs that also need changes, each paired with a precise natural-language instruction. The concurrent dispatch of these rewrites, described in Section 5.3.2.2, is implemented with Python's `ThreadPoolExecutor`, calling the **Block Rewrite agent** once per affected block. This can be observed in the following implementation excerpt (Listing 6.3):

```
    with concurrent.futures.ThreadPoolExecutor() as executor:
        futures = [
            executor.submit(
                self._rewrite_single_block,
                run_id,
                rewrite["block_id"],
                rewrite["instruction"],
                outline_text,
            )
            for rewrite in cross_block_rewrites
        ]

        for future in futures:
            future.result()
```

Listing 6.3: Parallel execution of cross-block rewrites

  This pattern keeps the Chat agent's prompt small while still propagating edits across the document.

- **Visual asset generation:** The Visual Generator tool, introduced in Section 5.3.2.2, detects whether the user asked for a quantitative comparison or a time series and renders the result as a Chart.js chart served through the QuickChart API. The resulting `visual_markdown` string is inserted directly into `updated_markdown` at the appropriate position, so no extra round-trip is needed from the frontend. Whenever a block update includes visual content, the backend scans the resulting Markdown for external chart URLs (QuickChart); each detected image is downloaded, persisted as a `WorkspaceFile`,

and its URL is replaced with a local backend download endpoint, so that charts remain accessible independently of the external service.

- **Impact propagation:** The asynchronous impact check described in Section 5.3.2.2 is carried out by the Block Impact service: after any block save, it computes a unified diff of the old and new content, queries all blocks with an incoming *references* or *depends_on* relationship to the changed block, and calls the Block Impact agent once per affected block. Suggestions are stored as pending `ImpactSuggestionRecord` entries and surfaced in the editor UI.

6.5 MCP Callback Channel

During generation and chat interactions, Mattin AI agents can call back into the backend in real time to read or modify the document state. This bidirectional channel is implemented through the Model Context Protocol (MCP), exposed as HTTP JSON-RPC 2.0 on the `mcp` endpoint, separate from the `api/v1` prefix.

The MCP handler in `server_handler.py` implements protocol version 2025-03-26. It accepts requests at both `mcp/v1/{app_slug}/{server_slug}` and `mcp/v1/id/{app_id}/{server_id}` to accommodate the two URL formats Mattin AI may use. The handler processes three JSON-RPC method categories: `initialize`, `tools/list`, and `tools/call`. The first returns server metadata and capabilities, the second returns the tool descriptors, and the third dispatches to the appropriate tool implementation.

At the routing level, the backend exposes both MCP entrypoints explicitly and forwards the request to a dedicated handler (Listing 6.4):

```
@router.post("/mcp/v1/id/{app_id}/{server_id}")
def mcp_endpoint_by_id(
    app_id: str,
    server_id: str,
    payload: dict[str, Any],
    request: Request,
    db: Session = Depends(get_db),
) -> dict[str, Any]:
    del app_id, server_id
    return handle_mcp_request(payload=payload, request=request, db=
        db)

@router.post("/mcp/v1/{app_slug}/{server_slug}")
def mcp_endpoint_by_slug(
    app_slug: str,
    server_slug: str,
    payload: dict[str, Any],
    request: Request,
    db: Session = Depends(get_db),
) -> dict[str, Any]:
    del app_slug, server_slug
    return handle_mcp_request(payload=payload, request=request, db=
        db)
```

Listing 6.4: MCP HTTP entrypoints exposed by the backend

A `_MCP_TOOL_ALIASES` dictionary maps shorthand names, such as `review_consistency` and `workspace.list_blocks`, to canonical tool names, allowing agents to use dot-notation or abbreviated forms without breaking the handler. This alias resolution is also explicit in the implementation (Listing 6.5):

```
_MCP_TOOL_ALIASES: dict[str, str] = {
```

```
    "review_consistency": "workspace_review_consistency",
    "workspace.list_blocks": "workspace_list_blocks",
    "workspace.get_block": "workspace_get_block",
    "workspace.review_consistency": "workspace_review_consistency",
    "workspace.create_block": "workspace_create_block",
    "workspace.delete_block": "workspace_delete_block",
}

def _call_tool(
    service: WorkspaceService,
    tool_name: str,
    arguments: dict[str, Any],
) -> dict[str, Any]:
    resolved_tool_name = _MCP_TOOL_ALIASES.get(tool_name, tool_name)
    ...
```

Listing 6.5: Alias resolution for MCP tool dispatch

An optional `mcp_server_token` can be set via environment variable; if configured, the handler validates the `x-mcp-token` header or `Authorisation: Bearer` token before dispatching any tool call. SSE transport is declared but returns HTTP 501, as the current integration uses HTTP transport exclusively.

The MCP channel is what allows generation agents to be truly document-aware: rather than receiving the entire document as a prompt, the agent can list blocks, read specific sections, and write back changes incrementally. This reduces prompt size and latency while enabling more precise, context-driven edits.

We can see the list of functions the MCP exposes to the agents in the table 6.1.

Tool	Purpose
`workspace_list_blocks`	Lists all blocks with their metadata for a given run.
`workspace_get_block`	Fetches one block with its full Markdown content.
`workspace_get_blocks_content`	Batch-fetches full content for a list of block IDs.
`workspace_create_block`	Creates a new block with optional positioning before or after a specific block.
`workspace_delete_block`	Deletes a block and reindexes the remaining blocks.
`workspace_review_consistency`	Checks for duplicate titles and broken Markdown links.
`workspace_get_block_relationships`	Returns semantic relationships for a given block.

Table 6.1: MCP tools exposed by the backend.

CHAPTER 7

System Evaluation

Testing in this project focused mainly on functional validation. This decision was motivated by the nature of the application, where the most relevant requirement was not only that isolated functions behaved correctly, but that the complete workflow operated as expected from the user's perspective. In practice, this involved validating end-to-end interactions such as creating a workspace, uploading contextual files, generating content, and editing the resulting blocks.

The main goal of the testing process was to ensure that the core features of the system behaved consistently, reliably, and reproducibly after each relevant change. To achieve this, the project combined manual validation during development with automated end-to-end functional tests, supported by a continuous integration workflow in GitHub Actions.

However, we also aim to test the non-functional requirements defined in Chapter 4, for these we applied different metrics depending on the necessities.

7.1 Functional Testing Approach

Functional testing aims to verify that the application satisfies the expected behaviour in realistic usage scenarios. Instead of focusing only on internal implementation details, functional tests validate the observable behaviour of the system through complete user-facing workflows.

In this case, the most important scenarios involved the main working flow of the application. The implemented tests cover actions such as:

1. Creating a new workspace.
2. Uploading source files or contextual material.
3. Generating a document from the uploaded context.

4. Creating new blocks inside the generated document.
5. Interacting with the chat system to request modifications to a block.
6. Deleting blocks and verifying that navigation and workspace state remain correct.

This approach is especially suitable for this project because the application depends on the coordinated interaction between several layers, including the frontend, backend, database, and external services. Functional testing makes it possible to detect integration problems that would not necessarily appear in isolated unit tests [9].

7.1.1 End-to-End Test Automation

To automate these checks, the project uses Playwright as the main end-to-end testing framework. Playwright makes it possible to simulate real user interactions with the web interface, including filling in forms, clicking buttons, navigating between pages, and uploading files.

The use of Playwright provides several advantages. First, it reduces the effort required to repeat the same tests after changes in the codebase. Second, it limits the risk of human error during manual verification. Third, it establishes a repeatable and objective way to validate whether the system still behaves correctly after new changes are introduced.

The implemented tests do not only verify whether interface elements are visible, but whether complete workflows are successfully executed. For example, the test suite checks that a workspace can be created, that uploaded context files appear correctly, that document generation leads to the expected editor view, and that block creation, chat interaction, and deletion work as intended.

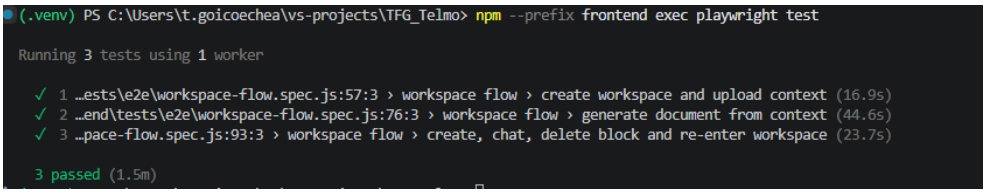


Figure 7.1: Execution of an automated Playwright functional test.

7.1.2 Continuous Integration with GitHub Actions

In addition to local automated testing, the project includes a continuous integration pipeline implemented with GitHub Actions[1]. The purpose of this workflow is to automatically run the functional test suite whenever relevant changes are pushed to the repository, helping to detect regressions as early as possible.

However, this pipeline is not executed on GitHub-hosted runners. Instead, it relies on a self-hosted GitHub Actions runner configured on a local machine. This design decision was

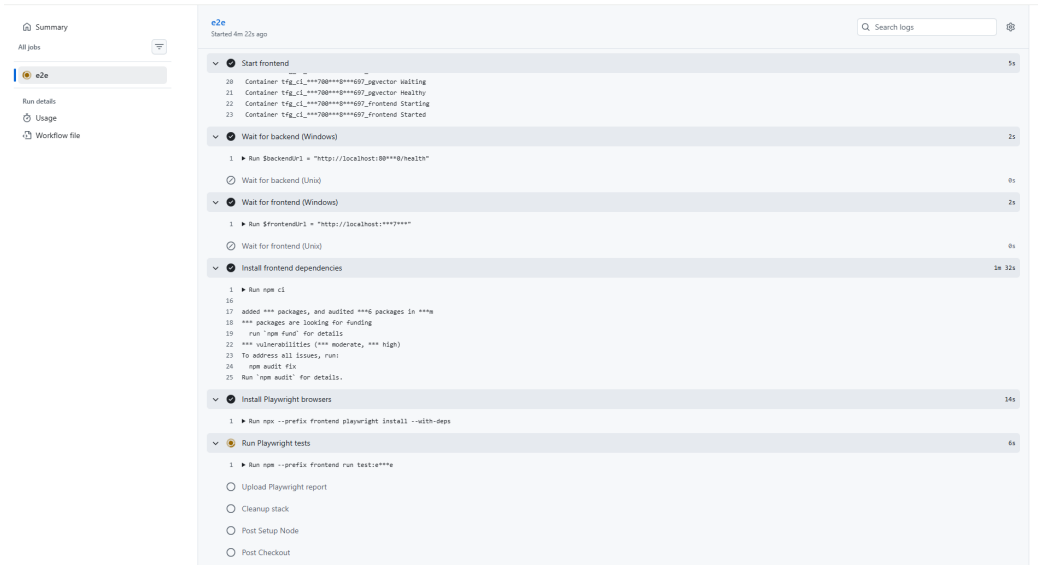
[1] https://github.com/features/actions

necessary because the application depends on Mattin, an external AI platform that, in the development and testing setup of this project, must be running locally and correctly configured before the workflow can complete successfully. Therefore, the automated execution depends on two infrastructure conditions: the self-hosted runner must be active and connected to the repository, and the local Mattin AI service must be available during the execution of the tests.

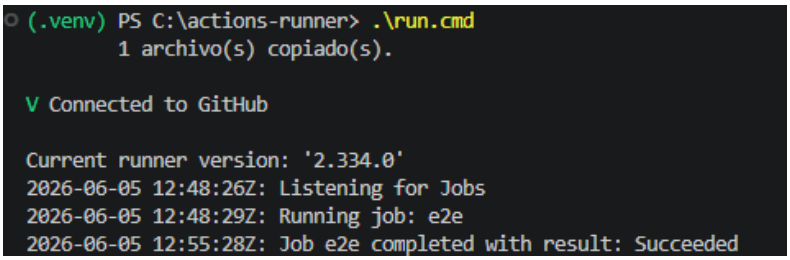
Once these prerequisites are satisfied, the configured workflow performs the following tasks:

1. It checks out the repository source code.
2. It prepares the execution environment.
3. It validates the required Mattin-related configuration and secrets.
4. It starts the required services using Docker Compose.
5. It runs the database migrations.
6. It starts both the backend and frontend services.
7. It waits until both services are available.
8. It executes the Playwright end-to-end tests.
9. It uploads test artifacts when failures occur.
10. It cleans up the environment after execution.

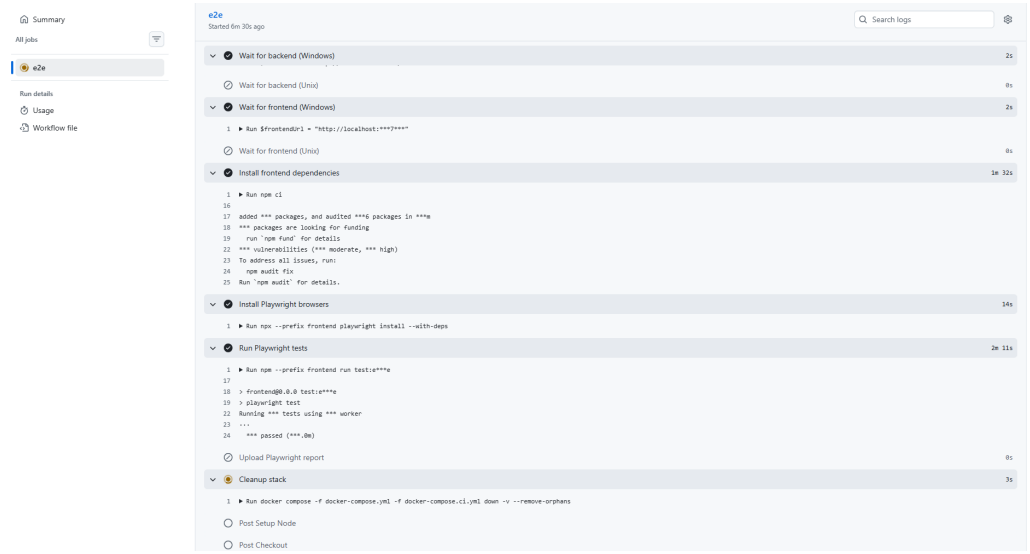
Figures 7.2a, 7.2b, and 7.2c illustrate the practical execution of this pipeline. They show the workflow running in GitHub Actions, the self-hosted runner listening for and completing the job, and the detailed execution steps of the end-to-end process.



(a) Overview of the *e2e* workflow execution in GitHub Actions.



(b) Self-hosted GitHub Actions runner connected locally and processing the job.



(c) Detailed workflow steps, including frontend startup, dependency installation, and Playwright test execution.

Figure 7.2: GitHub Actions end-to-end testing pipeline executed on a self-hosted runner with local infrastructure dependencies.

This pipeline ensures that the full application stack can be validated in a controlled and reproducible environment. Nevertheless, it is important to note that the workflow still depends on the availability of local supporting infrastructure. In other words, the tests are automatically triggered and executed through GitHub Actions, but their successful completion requires the machine hosting the self-hosted runner to have both the runner service active and the Mattin AI environment accessible.

When a failure occurs, the workflow preserves useful debugging information such as reports, screenshots, traces, or videos, which simplifies later analysis and diagnosis.

7.2 Non-Functional Testing

To address the non-functional requirements identified in Chapter 4, the project used different validation strategies depending on each requirement.

7.2.1 Security and Confidentiality

Security and confidentiality were not evaluated through a dedicated testing strategy in this project. This is mainly because, as mentioned throughout the project, the aim of the project changed from a production-ready product to an investigation project, and therefore the development did not include a security phase.

7.2.2 Performance

Performance was evaluated indirectly by observing the execution time of the main AI-related workflows. To address this, Mattin AI allows us to add a LangSmith API key, which enables us to trace the response times of every call to every agent configured in our app.

As declared in the requirements, for agents response time, we aimed to have an average response time below 10 seconds for minor block refinements, excluding the network latency associated with external LLM calls. Also, for consistency synchronisation, we intended to get a background detection and user notification in under 10 seconds. For both of these, we executed a request in an already created workspace and generated test document, and asked for a minimal change. We changed the name of a person in one block that also appeared in another block, and asked the agent of the second one to review and fix the name in case they did not match. The agent reviewed and offered a correct refactor of the block, and also detected correctly a dependence in another block, which we also changed. We can see in Figure 7.3 that all the times are below our constraints, thus meaning we meet the requirements specified for these metrics.

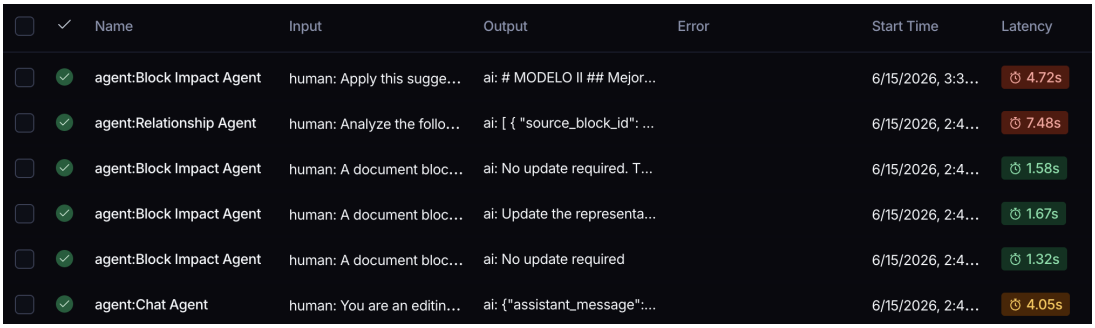


Figure 7.3: LangSmith tracing

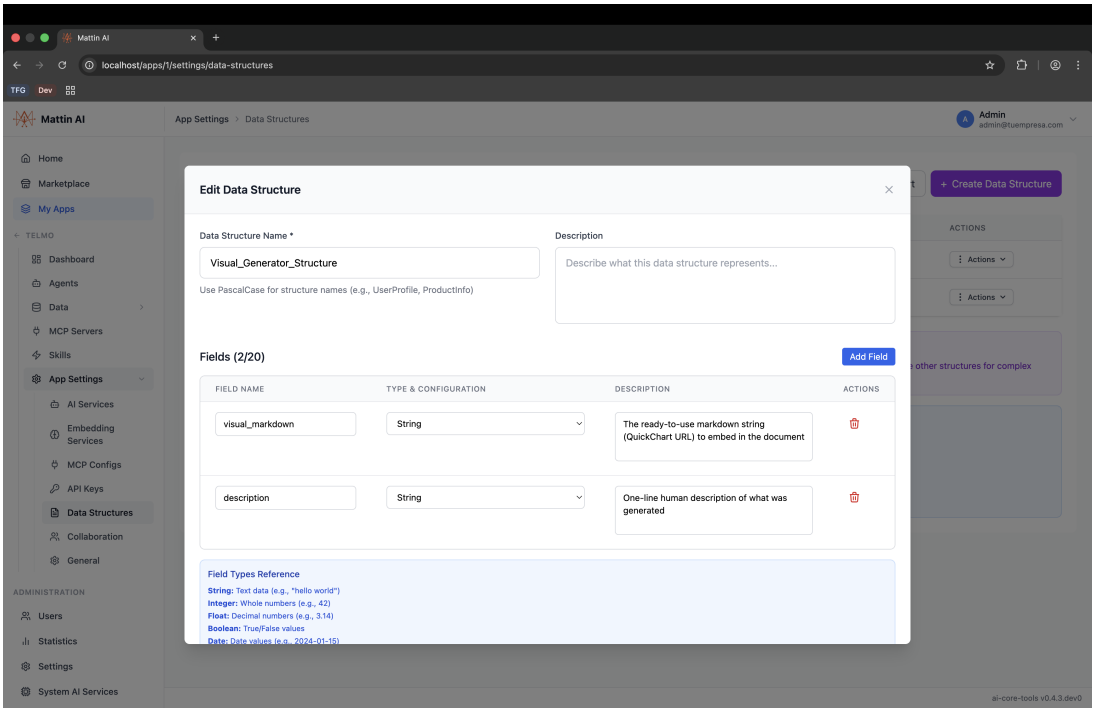


Figure 7.4: Data Structure Configuration in Mattin AI

7.2.3 Reliability

Reliability was addressed by validating the robustness of the AI-generated outputs and by ensuring that the system handled structured responses consistently. During agent configuration, Mattin AI allows the definition of a data structure that the model output must satisfy. We can see in Figures 7.4 and 7.5 how we configured the data structure for the visual generator agent.

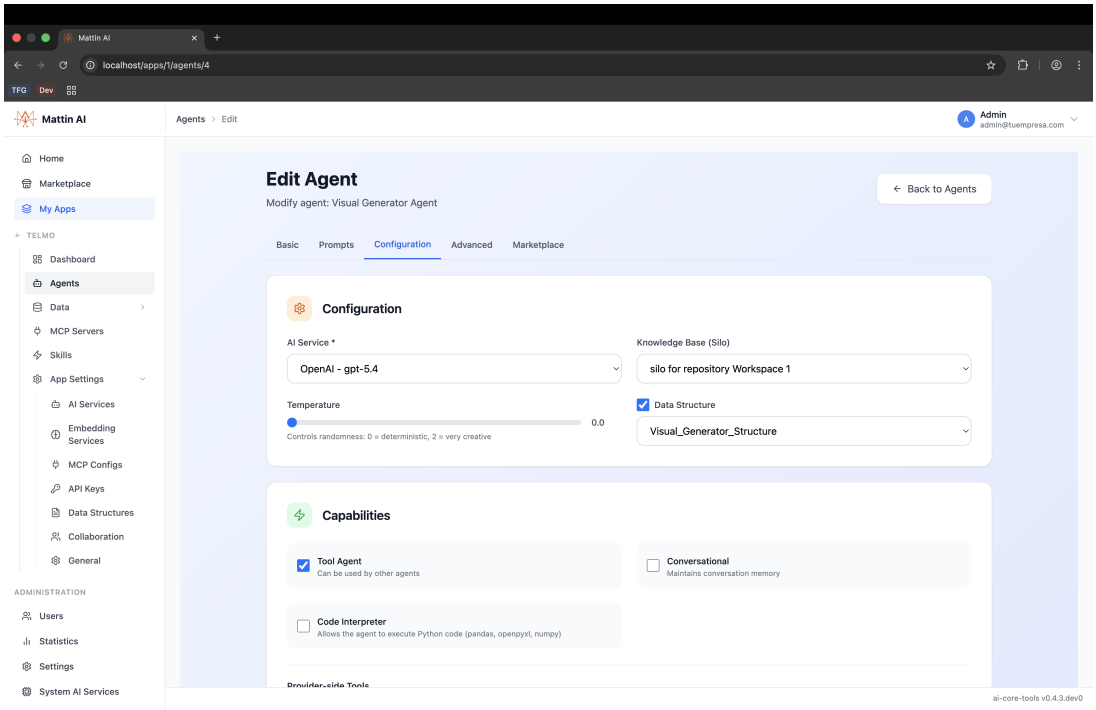


Figure 7.5: Agent Configuration in Mattin AI

7.2.4 Maintainability and Deployment

Maintainability and deployment were supported through containerisation. The complete system was packaged using Docker Compose, with separate containers for the database, the backend, and the frontend. This structure isolates each component while preserving a consistent execution environment across development and deployment.

The backend container installs its Python dependencies through Poetry and starts the application together with the required database migrations. The frontend is also containerised with its own Node.js environment, and the database runs in a dedicated PostgreSQL container with pgvector support and persistent storage. By defining the service configuration, ports, environment variables, and internal dependencies in a single compose file, the full stack can be started in a reproducible way with minimal setup effort. The code of the *docker-compose.yml* can be recognised in the Listing 7.1

```
services:
  db:
    image: pgvector/pgvector:pg16
    container_name: tfg_pgvector
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-tfg_user}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-tfg_password}
      POSTGRES_DB: ${POSTGRES_DB:-tfg_db}
```

```
    ports:
      - "${POSTGRES_PORT:-5432}:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-tfg_user} -d ${
          POSTGRES_DB:-tfg_db}"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: tfg_backend
    depends_on:
      db:
        condition: service_healthy
    env_file:
      - ./backend/.env
    environment:
      DATABASE_URL: postgresql+psycopg://tfg_user:tfg_password@db:5432/
          tfg_db
      POSTGRES_HOST: db
      POSTGRES_PORT: 5432
    ports:
      - "8010:8010"
    volumes:
      - ./backend:/app
    extra_hosts:
      - "host.docker.internal:host-gateway"
    command: /bin/sh -c "python -m alembic upgrade head && python -m
        uvicorn app.main:app --host 0.0.0.0 --port 8010 --reload"

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: tfg_frontend
    depends_on:
      - backend
    environment:
      VITE_API_BASE_URL: ${VITE_API_BASE_URL:-http://localhost:8010/api/v1
          }
      VITE_AUTH_USER_STORAGE_KEY: ${VITE_AUTH_USER_STORAGE_KEY:-
          tfg_auth_user}
      CHOKIDAR_USEPOLLING: "true"
    ports:
      - "5173:5173"
    volumes:
      - ./frontend:/app
      - frontend_node_modules:/app/node_modules
```

```
58      command: npm run dev -- --host 0.0.0.0 --port 5173
59
60  volumes:
61    postgres_data:
62    frontend_node_modules:
```

Listing 7.1: Docker Compose

7.3 Benefits of the Adopted Strategy

The combination of functional testing and continuous integration has provided important benefits to the project. On the one hand, it has made it possible to validate the application through realistic user scenarios rather than only through isolated checks. On the other hand, it has introduced an automated quality control mechanism that increases confidence in the stability of the system as development progresses.

This was particularly important given the architecture of the application, where several components must work together correctly. In such a context, a feature may appear correct in isolation but fail when executed as part of the complete workflow. End-to-end functional testing is especially valuable in detecting this kind of issue.

Moreover, the inclusion of GitHub Actions improves the maintainability of the project, since regressions can be identified early and the main branch can be protected with automated validation.

7.4 Limitations and Future Work

Although the adopted testing strategy has been effective for validating the main workflow of the application, it also has some limitations. Functional end-to-end tests are generally slower and more expensive to maintain than unit tests [9], since they depend on the whole application stack and on the correct behaviour of the surrounding environment. In addition, when external services are involved, some failures may be caused by factors outside the application itself.

In this project, the continuous integration pipeline also depends on the availability of a locally running Mattin AI instance and an active self-hosted GitHub Actions runner. As a result, the testing workflow is automated, but not fully independent from the local execution environment. This means that infrastructure availability must be considered as part of the validation process.

As future work, it would be advisable to reduce this operational dependency by moving towards a more isolated and reproducible CI environment, or by introducing mocks or testing doubles for external AI-dependent functionality. In parallel, the current functional testing strategy could be complemented with a larger set of unit and integration tests for specific backend services and relevant frontend components.

7.5 Conclusion

In conclusion, testing in this project has relied mainly on automated functional validation of the main end-to-end workflows of the application. This testing strategy has been reinforced through a GitHub Actions continuous integration pipeline that automatically executes the test suite in a reproducible environment. Together, these mechanisms have improved the reliability of the system, reduced the risk of regressions, and established a solid basis for future development.

Our primary objective was to ensure the application's functionality and to test the system exactly as we had intended; given the positive response, we can conclude that the evaluation was appropriate and successful.

CHAPTER 8

Monitoring and Control

This chapter documents all changes and deviations that have occurred, and compares them to the original planning.

8.1 Scope Management

Given its iterative and incremental nature, the project has undergone changes in terms of its scope. These changes were discussed and addressed during meetings with the project tutors. Apart from the odd task being added or removed, the most significant change was undoubtedly the shift in the project's scope, from a more production-oriented approach to a project entirely focused on experimentation and research, removing entire iterations and adding new ones. It should also be borne in mind that there was a significant underestimation in the calculation of time required to draft the report, which reduced time for more implementations. These changes implied the following consequences:

- **Iteration changes:** Original planning considered a third iteration based on users and document management, whilst the fourth iteration included a document exporting system and a better user interface. All these focused on having the best user experience possible, but our aim was to implement the most features possible using agents rather than making a product. The third iteration happened in a much simpler way, user management had no deep session handling nor user control. On the other hand, during the fourth iteration, the interface was updated, but not for user feedback, but to add features such as a Markdown visualizer. We also upgraded and reviewed the agents system, adding new features as the visual generator agent, capable of generating charts and diagrams.
- **Time allocated for the report:** At the beginning of the project, 70 hours were planned for writing the bachelor thesis. However, due to an initial underestimation of the effort

required, the actual time spent on the report increased significantly and ultimately exceeded 100 hours. As a result, the time initially reserved for implementing further improvements had to be redirected toward completing the bachelor thesis.

- **Testing handling:** Instead of working on testing after every iteration, we moved all the testing to the end, and decided just to work on end-to-end validation, without performance evaluation, in-web deployment, or multi-user scenarios.

8.2 Time Tracking

As mentioned earlier, deviations were managed by redistributing tasks. These adjustments involved changes in both deadlines and working hours.

8.2.1 Deadlines

We can see in the Table 8.1 the real dates for the deadlines, where we can appreciate a very short third iteration, which threw off the original deadlines. It is to mention that during the end of the fourth iteration we already started to use a great amount of time to the writing of the report (more or less from the 25th of may onwards).

Milestone	Estimated Date	Real Date
End of 1st milestone	10/04/2026	10/04/2026
End of 2nd milestone	01/05/2026	07/05/2026
End of 3rd milestone	22/05/2026	12/05/2026
End of 4th milestone	05/06/2026	02/06/2026
End of thesis and project	14/06/2026	14/06/2026
Defense	T.B.D.	T.B.D.

Table 8.1: Estimated and Real Dates of Milestones.

Although further improvements would have been possible to implement, it was decided to focus efforts on writing this report, given that the project's objectives had been achieved and that we wanted to ensure the report's completion, decision that was proved right when we understood that we underestimated how long it would take.

8.2.2 Task Dedication

We compared the task estimation table with the real dedication to each task in order to obtain the real deviations. We can see this in Figure 8.1. We can appreciate three major deviations:

- With the elimination of testing in the second and third iterations, and simplification of user and document management tasks, we see a reduction of hours in comparison to the estimation.
- As we started the fourth iteration earlier, we spent more time than planned on the original tasks.

- The greatest deviation, the report writing estimate, with an almost 50 hour deficit.

Regardless of the changing and reorganisation of tasks, we only have a 4 hour deviation in terms of the totality of the project, which is a deviation of just over one per cent, which allows us to consider that we made a good management of hours even though we had to adapt throughout our first great project.

Iteration	Task	Estimated Hours	Substitution Task	Real Dedication (Hours)	Difference
1st Iteration	Technology Exploration and Learning (TL)	10		10	0
	Requirement Analysis (RA)	20		20	0
	Scope Definition (SD)	10		10	0
	Architecture Design (AD)	20		20	0
	Total 1st Iteration	60		60	0
2nd Iteration	Content Generation (CG)	40		40	0
	Block Editing System (BE)	35		35	0
	Inter-Block Dependency System (ID)	40		30	-10
	Initial Testing (T)	10		0	-10
	Total 2nd Iteration	125		105	-20
3rd Iteration	User Management (UM)	10		3	-7
	Document Management (DM)	30		6	-24
	Testing (T)	10		0	-10
	Total 3rd Iteration	50		9	-41
4th Iteration	Document Export System (EX)	15	Configuration Extras (Mattin connections, github agents, docker, ...)	24	9
	User Interface Refinement (UI)	10	User Interface Refinement (UI)	10	0
	System Validation and Deployment (VD)	15	Agents System Extension + Functional Testing	24	9
	Total 4th Iteration	40	Total 4th Iteration	58	18
Management	Planning (PL)	20	Planning (PL)	20	0
	Monitoring and Control (MC)	10	Monitoring and Control (MC)	10	0
	Total Management	30	Total Management	30	0
Academic Work	Final Report (FR)	70	Final Report (FR)	117	47
	Defense (D)	10	Defense (D)	10	0
	Total Academic Work	80	Total Academic Work	127	47
TOTAL	-	385	-	389	4

Figure 8.1: Task Dedication Table

CHAPTER 9

Conclusions

This chapter closes the project by summarising its main outcomes, reviewing the degree to which the initial objectives were achieved, reflecting on the main lessons learned during development, and outlining several directions for future work.

9.1 Overall Conclusions

This project has demonstrated that a multi-agent approach can be effectively applied to professional document generation and block-level editing. The resulting system is not limited to producing a single static output; instead, it supports an iterative workflow in which a document is first generated from a prompt and contextual material, then divided into editable blocks, and finally refined through local or document-wide AI-assisted interactions.

One of the main contributions of the project is the combination of agent orchestration with a structured editing model. Rather than treating the generated document as an opaque response, the system stores it as a set of persistent blocks with summaries, relationships, and chat history. This makes it possible to support more controlled and transparent editing workflows, where users can modify individual parts of a document while preserving the overall structure.

Another relevant contribution is the integration of agents with the Model Context Protocol (MCP). Through this mechanism, agents are not restricted to passive text generation, but can access workspace information and participate in document operations in a grounded way. In practice, this has enabled the implementation of workflows such as context-aware generation, semantic relationship detection, cross-block rewrites, document-wide editing, and visual asset generation.

From an engineering perspective, the project has also shown that this type of system can be implemented with a reasonably modular architecture. The separation between frontend, backend, persistence layer, AI orchestration, and MCP-exposed tools has made the application

easier to reason about and extend. This modularity is especially relevant in a context where the behaviour of AI systems is still evolving rapidly and where replacing or refining individual agent pipelines may become necessary.

The evaluation carried out in the project, mainly through automated end-to-end functional testing, supports the conclusion that the main workflow of the application is operational. The implemented validation process confirms that users can create workspaces, upload contextual files, generate documents, edit blocks, and interact with the AI-assisted editing features in a consistent way. Although this does not constitute an exhaustive quality evaluation of the generated content, it does provide evidence that the system works correctly as an integrated software product.

Overall, the project can be considered successful. It delivers a functional prototype that validates the feasibility of AI-assisted document generation based on multiple specialised agents, while also providing a solid technical basis for future refinement and experimentation.

9.2 Project Objectives Review

The project objectives defined at the beginning of the work have been achieved to a satisfactory degree:

- **Study of AI-assisted document generation systems:** This objective was addressed through the analysis of the technological context of LLM-based applications, agentic workflows, and MCP-based integrations, which informed the final design of the system.
- **Design of a multi-agent architecture for document generation and editing:** This objective was fulfilled by defining a system composed of specialised agents with clearly separated responsibilities, including document writing, document splitting, block chat, block rewriting, block impact analysis, relationship detection, document-wide editing, and visual generation.
- **Implementation of a functional web application:** The project resulted in a working application with workspace management, contextual file upload, AI-assisted document generation, persistent block storage, block-level editing, and support for structural operations such as block creation, deletion, and document-wide updates.
- **Integration of AI agents with grounded tooling:** This objective was achieved through the incorporation of MCP-compatible tools that allow agents to inspect and manipulate workspace state in a controlled way, bridging the gap between text generation and application logic.
- **Validation of the system behaviour:** The project includes automated end-to-end tests and a continuous integration workflow that verify the correctness of the main user-facing workflows. Even if the evaluation was mainly functional rather than experimental, it provides a reliable baseline for the implemented features.

Even though some originally envisioned production-oriented features were simplified or postponed, the core objective of building and validating a multi-agent document assistant has been achieved.

9.3 Lessons Learned

One of the main lessons learned during the development of this project concerns scope management. The project evolved from a more product-oriented idea towards a more experimental and research-focused implementation. This change was not a failure in planning, but rather an adaptation to the realities of the available time, the complexity of the AI-related features, and the academic objectives of the thesis. In hindsight, prioritising the most distinctive technical components of the system was the correct decision.

Another important lesson is that AI-centred applications require a different development mindset from conventional software systems. The challenge is not only to implement the surrounding infrastructure, but also to define robust interfaces between deterministic code and probabilistic agent behaviour. This was particularly visible in areas such as structured JSON validation, block dependency analysis, cross-block consistency handling, and the routing between local and document-wide editing requests.

The project also highlighted the importance of modularity when working with agent-based systems. By separating responsibilities between agents and keeping the backend in control of orchestration, validation, and persistence, the system became easier to evolve than a monolithic prompt-driven approach would have been. This architectural decision reduced the impact of changes in prompts, agent responsibilities, and workflow design.

Finally, the project provided a practical lesson in time estimation. While the technical development was managed with acceptable overall deviation, the effort required to write the report was clearly underestimated. This had a direct impact on the number of additional features and evaluation activities that could be completed. As a result, one of the clearest takeaways is that documentation and thesis writing must be treated as a major work package from the beginning, not as a final-stage activity.

9.4 Future Work

Several lines of future work emerge naturally from the current state of the project:

- **Evaluation of content quality:** The current validation confirms that the application works functionally, but it does not measure in depth the quality, usefulness, or professional adequacy of the generated documents. A future study could compare outputs across document types, prompts, and models, and include human evaluation to assess coherence, correctness, and editing efficiency.
- **More robust dependency and consistency management:** The current relationship and impact analysis already provide valuable support for cross-block editing, but these

mechanisms could be improved with richer semantic modelling, better conflict detection, and more explainable rewrite suggestions.

- **Stronger testing strategy:** The project would benefit from complementing the current end-to-end tests with more unit and integration tests, especially for backend services, persistence logic, and failure handling around external AI calls. In addition, the CI pipeline could be made more reproducible by reducing its dependency on a locally available Mattin AI instance.
- **Deployment and multi-user maturity:** Although the prototype supports the core workflow, future iterations could strengthen user management, concurrent editing scenarios, permissions, and deployment in a more production-ready environment.
- **Export and collaboration capabilities:** The system could be extended with advanced export formats, revision history visualisation, collaborative review workflows, and mechanisms for tracking the origin and justification of AI-generated changes.
- **Extension of the agent ecosystem:** Another natural continuation would be to explore additional specialised agents, such as agents for style normalisation, citation assistance, compliance checking, or domain-specific document generation. This would further test the scalability of the proposed architecture.

In summary, the project has established a solid foundation for AI-assisted professional document authoring. While the current implementation should be understood as a functional prototype rather than a finished commercial product, it successfully proves that multi-agent orchestration, grounded tooling, and block-based editing can be combined into a coherent and useful document workflow.

APPENDIX A

Declaration of use of generative AI

more information: https://www.ehu.eus/en/web/adimen-artifiziala/home

Tool/s used:

- **Github Copilot:** LKS Next provided a company account to use Github Copilot's Business Plan.
- **OpenAI's DALL-E**[1]**:** image generator, built into ChatGPT's chat.
- **Google's Stitch**[2]**:** UI design generator.

Purpose of use (tick as appropriate):

☒ Text generation

☐ Reformulation

☐ Translation / proofreading

☒ Structural suggestion

☐ Methodological support

☒ Help with coding/programming

☒ Other

[1] https://chatgpt.com/g/g-2fkFE8rbu-dall-e

[2] https://stitch.withgoogle.com/

Description of the use: In order to help with the programming and structuration of the project, the company handed us the Github Copilot account. As mentioned in the subsection 5.4.5, we developed a system of custom agents so that we could use more efficiently the help of these tools. We also used the same tools to summarise faster all the project's information, previous to the writing of this document, to help us plan how to structure it. During the writing we also used Github Copilot in order to help us explain the most technical pieces of text in the clearest of ways. Before designing the UI, we used Google's Stitch tool to help us design the UI for the application and get some ideas. And lastly, we used OpenAI's image generator agent, in order to create some figures, to visualise some of the agents pipelines, explained throughout the document.

Authorship commitment:
I confirm that all generated content has been personally reviewed, reworked and validated by me.

Bibliography

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017. See pages 1, 3, and 5.

[2] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022. See pages 1, 3, 5, and 6.

[3] Zijian Zhang, Fangshi Du, Xingjian Liu, Pan Chen, Oliver Huang, Runlong Ye, Michael Liut, and Alán Aspuru-Guzik. TreeWriter: AI-assisted hierarchical planning and writing for long-form documents. *arXiv preprint arXiv:2601.12740*, 2026. See pages 1, 6.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems 33*, 2020. See pages 4, 24.

[5] Mina Lee, Percy Liang, and Qian Yang. CoAuthor: Designing a human-AI collaborative writing dataset for exploring language model capabilities. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, 2022. See page 6.

[6] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, and Dave Thomas. Manifesto for agile software development. https://agilemanifesto.org/, 2001. See page 10.

[7] European Parliament and Council of the European Union. Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation). Technical Report L 119, Official Journal of the European Union, 2016. https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679. See page 20.

[8] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 2025. See page 31.

[9] Swetha Talakola. Automated End-to-End Testing with Playwright for React Applications. *International Journal of Emerging Research in Engineering and Technology*, 5(1):38–47, 2024. See pages 50, 57.